POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Informatyki
Filia w Gdańsku

Otrombke Natalia
Nr albumu s26828
Nazwa specjalizacji: Cyberbezpieczeństwo

Kasprowicz Oliwier
Nr albumu s30290
Nazwa specjalizacji: Cyberbezpieczeństwo

Gutowski Michał
Nr albumu s30572
Nazwa specjalizacji: Cyberbezpieczeństwo

Parkitny Miłosz
Nr albumu s30287
Nazwa specjalizacji: Cyberbezpieczeństwo

Wronisz Wojciech
Nr albumu s26999
Nazwa specjalizacji: Cyberbezpieczeństwo

RacePortal — platforma wydarzeń motorsportowych

Rodzaj pracy
inżynierska

Imię i nazwisko promotora
dr hab. Marek Bendarczyk

Gdańsk, miesiąc, rok obrony

Streszczenie: Praca przedstawia projekt i implementację systemu RacePortal — platformy wspierającej katalogowanie wydarzeń motorsportowych oraz proces zapisów uczestników. System obejmuje aplikację webową, backend REST oparty o Spring Boot i MySQL oraz aplikację mobilną Expo pełniącą rolę publicznej wizytówki katalogu. W pracy opisano problem dziedzinowy, wymagania, architekturę, realizację kluczowych modułów (wydarzenia, garaż, zgłoszenia, role), mechanizmy bezpieczeństwa (JWT, RBAC) oraz weryfikację testami automatycznymi. Wynikiem jest działające, konteneryzowane środowisko demonstracyjne zgodne z zakresem MVP projektu.
Słowa kluczowe: motorsport, katalog wydarzeń, system zapisów, zarządzanie wydarzeniami, aplikacja mobilna

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: RacePortal — platforma wydarzeń motorsportowych **Temat projektu po angielsku:** RacePortal — a platform for motorsports events	**Akronim:** RP **Data ustalenia tematu** 2025-10-01
Promotor: dr hab. Marek Bendarczyk	**Konsultanci:** 1. — brak —

Cele projektu:
Celem projektu jest zaprojektowanie i zaimplementowanie systemu RacePortal — centralnego katalogu wydarzeń motorsportowych w Polsce, umożliwiającego przeglądanie wydarzeń (lista, mapa, kalendarz), zarządzanie garażem pojazdów, zgłoszenia udziału oraz obsługę ról organizatora i administratora, wraz z aplikacją mobilną typu wizytówka oraz konteneryzowanym środowiskiem uruchomieniowym.

Rezultaty projektu:
Aplikacja webowa (React/Vite), API REST (Spring Boot) z bazą MySQL, aplikacja mobilna Expo (katalog wydarzeń z CTA do weba), wspólne typy API (`packages/api-types`), dokumentacja projektowa oraz zestaw testów automatycznych (JUnit, Vitest, Playwright).
Katalog wydarzeń z filtrami oraz widokami lista/mapa/kalendarz; szczegóły wydarzenia i wyznaczanie trasy (OSRM); konta użytkowników (JWT, RBAC); garaż pojazdów i dopasowanie kategorii do wydarzenia (`carMatch`); system zgłoszeń ze statusami i powiadomieniami e-mail; panele organizatora i administratora; archiwum; mobilna wizytówka z przekierowaniem zapisu na web.

Miary sukcesu:
Działający stos Docker Compose (web, API, MySQL, backup) z SMTP Mailbux; realizacja funkcji MVP katalogu, zapisów i ról; pozytywne wyniki testów automatycznych API/web/mobile unit; możliwość demonstracji pełnego scenariusza: gość → kierowca → organizator → administrator oraz mobilka → CTA web.

Ograniczenia:
Brak natywnej bramki płatności (PSP); lokalny upload obrazów/PDF bez object storage (S3/MinIO); aplikacja mobilna bez pełnej parity z webem (świadoma wizytówka); brak publikacji w sklepach App Store/Google Play; brak formalnego testu obciążenia 10k rekordów / 50 RPS; produkcyjny TLS poza domkniętym zakresem lokalnego MVP (w prod: Cloudflare Tunnel).

Wykonawcy	**Numer albumu**	**Specjalizacja**	**Tryb studiów**
Otrombke Natalia	s26828	Cyberbezpieczeństwo	Stacjonarny
Kasprowicz Oliwier	s30290	Cyberbezpieczeństwo	Stacjonarny
Gutowski Michał	s30572	Cyberbezpieczeństwo	Stacjonarny
Parkitny Miłosz	s30287	Cyberbezpieczeństwo	Stacjonarny
Wronisz Wojciech	s26999	Cyberbezpieczeństwo	Stacjonarny

Data ukończenia projektu:	**Recenzent:**
7 listopada 2026	mgr Alex Polin

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Otrombke Natalia**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26828**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Kasprowicz Oliwier**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30290**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Gutowski Michał**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30572**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Parkitny Miłosz**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30287**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Wronisz Wojciech**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26999**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

Spis treści

1 Informacje wstępne **23**
1.1 Cel i zakres pracy 23
1.2 Wizja rozwiązania 24
1.3 Struktura pracy 24

2 Słownik pojęć **27**

3 Opis problemu **28**
3.1 Charakterystyka rynku wydarzeń motorsportowych 28
3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników 28
3.3 Problemy informacyjne po stronie kierowców i organizatorów 29
3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal 29
3.5 Aspekty techniczne i organizacyjne problemu 30

4 Kontekst biznesowy **31**
4.1 Segmentacja użytkowników 31
4.2 Interesariusze systemu 33
4.3 Propozycja wartości systemu 34
4.4 Kanały komunikacji i dystrybucji 36
4.5 Kluczowe zasoby i działania 36
4.6 Model działania platformy 37
4.7 Realistyczność wdrożenia i dalsze perspektywy 37
4.8 Kontekst prawny i etyczny 38

5 Analiza SWOT **40**
5.1 Mocne strony (Strengths) 40
5.2 Słabe strony (Weaknesses) 40
5.3 Szanse (Opportunities) 40
5.4 Zagrożenia (Threats) 41

6 Planowanie projektu **42**
6.1 Cele projektu 42

6.2 Zakres projektu . . . 42
6.3 Metodyka pracy . . . 42
6.4 Organizacja zespołu . . . 42
6.5 Harmonogram pracy . . . 42
6.6 Infrastruktura i środowisko pracy . . . 43
6.7 Analiza ryzyka projektowego . . . 43
6.7.1 Analiza zagrożeń . . . 43

7 Analiza wymagań **44**
7.1 Źródła i sposób pozyskania wymagań . . . 44
7.2 Wymagania ogólne i dziedzinowe . . . 45
7.3 Wymagania funkcjonalne . . . 46
7.4 Wymagania niefunkcjonalne . . . 48
7.5 Wymagania bezpieczeństwa . . . 49
7.6 Wymagania interfejsu użytkownika . . . 50
7.7 Diagramy UML . . . 51
7.7.1 Diagram przypadków użycia . . . 51
7.7.2 Diagram sekwencji: rejestracja i logowanie . . . 51
7.7.3 Diagram sekwencji: zapis na wydarzenie . . . 51
7.7.4 Diagram sekwencji: zarządzanie garażem . . . 51

8 Projekt systemu **52**
8.1 Założenia architektoniczne . . . 52
8.2 Architektura logiczna systemu . . . 52
8.3 Architektura fizyczna i wdrożeniowa . . . 52
8.4 Projekt bazy danych . . . 52
8.4.1 Diagram klas . . . 52
8.5 Projekt interfejsu REST API . . . 53
8.6 Projekt warstwy webowej . . . 54
8.6.1 Prototyp UI w Figma Make . . . 54
8.7 Projekt aplikacji mobilnej . . . 54
8.8 Projekt mechanizmów bezpieczeństwa . . . 55

9 Realizacja **56**
9.1 Organizacja implementacji . . . 56
9.2 Podział systemu na komponenty . . . 56
9.2.1 Backend . . . 56
9.2.2 Frontend webowy . . . 57
9.2.3 Aplikacja mobilna . . . 57

9.3 Integracja komponentów . . . 57
9.4 Konteneryzacja i uruchamianie środowiska . . . 57

10 Szczegóły implementacji backendu 58
10.1 Struktura projektu backendowego . . . 58
10.2 Model domenowy i encje . . . 59
10.3 Rejestracja użytkowników i logowanie . . . 60
10.4 Zarządzanie profilami użytkowników . . . 61
10.5 Obsługa wydarzeń . . . 62
10.6 Obsługa zgłoszeń . . . 63
10.7 Obsługa garażu . . . 64
10.8 Panel administratora i organizatora . . . 64
10.9 Integracja z bazą danych . . . 65
10.10 Obsługa wyjątków i walidacja . . . 66

11 Szczegóły implementacji frontendu webowego 68
11.1 Struktura aplikacji webowej . . . 68
11.2 Strona główna i katalog wydarzeń . . . 68
11.3 Widok szczegółów wydarzenia . . . 68
11.4 Logowanie i rejestracja . . . 68
11.5 Dashboard użytkownika . . . 68
11.6 Widok garażu . . . 68
11.7 Panel organizatora . . . 69
11.8 Panel administratora . . . 69
11.9 Mapa i elementy lokalizacyjne . . . 69
11.10 Responsywność i doświadczenie użytkownika . . . 69

12 Szczegóły implementacji aplikacji mobilnej 70
12.1 Założenia aplikacji mobilnej . . . 70
12.2 Architektura rozwiązania mobilnego . . . 70
12.3 Lista wydarzeń i widok szczegółów . . . 70
12.4 Nawigacja i interfejs użytkownika . . . 70
12.5 Integracja z API . . . 70
12.6 Przekierowanie do wersji webowej . . . 70
12.7 Ograniczenia wersji mobilnej . . . 71

13 Bezpieczeństwo systemu 72
13.1 Założenia bezpieczeństwa . . . 72
13.2 Klasyfikacja danych . . . 72
13.3 Uwierzytelnianie użytkowników . . . 72

13.4 Autoryzacja i kontrola dostępu RBAC . . . 72
13.5 Wykorzystanie tokenów JWT . . . 72
13.6 Walidacja danych wejściowych . . . 72
13.7 Ochrona zasobów użytkowników . . . 72
13.8 Bezpieczeństwo komunikacji . . . 73
13.9 Analiza zagrożeń . . . 73
13.10 Analiza ryzyka . . . 73
13.11 Ograniczenia przyjętych zabezpieczeń . . . 73

14 Testowanie i weryfikacja systemu 74
14.1 Strategia testowania . . . 74
14.2 Środowisko testowe . . . 74
14.3 Testy jednostkowe . . . 74
14.4 Testy integracyjne backendu . . . 74
14.5 Testy End-to-End aplikacji webowej . . . 74
14.6 Testy aplikacji mobilnej . . . 74
14.7 Testy mechanizmów bezpieczeństwa . . . 74
14.8 Automatyzacja uruchamiania testów . . . 75
14.9 Analiza wyników testów . . . 75
14.10 Ograniczenia procesu testowania . . . 75

15 Prezentacja systemu 76
15.1 Prezentacja głównych funkcjonalności . . . 76
15.2 Scenariusz użycia z perspektywy kierowcy . . . 76
15.3 Scenariusz użycia z perspektywy organizatora . . . 76
15.4 Scenariusz użycia z perspektywy administratora . . . 76
15.5 Podsumowanie działania systemu . . . 76

16 Nakład pracy 77
16.1 Liczba godzin przeznaczona na realizację projektu . . . 77
16.2 Podział zadań w zespole . . . 77
16.3 Narzędzia wspierające zarządzanie pracą . . . 77
16.4 Problemy organizacyjne i ich rozwiązanie . . . 77

17 Wkład własny 78
17.1 Natalia Otrombke . . . 78
17.2 Oliwier Kasprowicz . . . 78
17.3 Michał Gutowski . . . 78
17.4 Miłosz Parkitny . . . 79
17.5 Wojciech Wronisz . . . 79

18 Podsumowanie **80**
18.1 Największe wyzwania i sposób ich rozwiązania 80
18.2 Zmiany i adaptacje w trakcie realizacji 80
18.3 Ograniczenia końcowej wersji systemu 80
18.3.1 Ograniczenia wydajnościowe (RPS) 82
18.4 Możliwości dalszego rozwoju 82
18.5 Wnioski końcowe 82

Bibliografia **83**

Spis rysunków **84**

Spis tabel **85**

Załączniki **86**

Rozdział 1

Informacje wstępne

1.1 Cel i zakres pracy

Celem pracy jest zaprojektowanie oraz implementacja systemu **RacePortal** — platformy internetowej (z uzupełnieniem mobilnym) służącej jako centralny katalog wydarzeń motorsportowych w Polsce oraz narzędzie do obsługi zapisów kierowców. Projekt stanowi odpowiedź na rosnące zapotrzebowanie społeczności motorsportowej na ujednolicone narzędzie eliminujące rozproszenie informacji na grupach społecznościowych i niespójnych stronach organizatorów.

Zakres projektu obejmuje:

- **Aplikację webową** — kompleksowy interfejs użytkownika zrealizowany w architekturze SPA (React/Vite), obsługujący moduły katalogu wydarzeń, kont użytkowników, wirtualnego garażu pojazdów, mechanizmu dopasowania kategorii (`carMatch`), zgłoszeń na imprezy oraz dedykowane panele zarządzania dla organizatorów i administratorów.
- **API REST oraz warstwę danych** — backend wykonany w oparciu o framework Spring Boot oraz relacyjną bazę danych MySQL (z wydzielonymi strukturami encji i wspólnymi typami API w pakiecie `packages/api-types`), zapewniający bezpieczne uwierzytelnianie (JWT) oraz kontrolę dostępu opartą o role (RBAC: `USER`, `ORGANIZER`, `ADMIN`).
- **System powiadomień i map** — zautomatyzowane powiadomienia e-mail (realizowane m.in. przez serwer SMTP Mailbux) dotyczące potwierdzeń rejestracji, statusów zgłoszeń oraz weryfikacji kont, a także integrację map wydarzeń z usługą wyznaczania tras (domyślnie OSRM z możliwością rozszerzenia o Google Maps API).
- **Aplikację mobilną (Expo)** — lekką, wieloplatformową aplikację mobilną opartą na ekosystemie Expo, pełniącą rolę mobilnej wizytówki katalogu z czytelnym mechanizmem konwersji (Call to Action — CTA) przekierowującym użytkownika do pełnego serwisu webowego.

- **Infrastrukturę i testowanie** — pełną konteneryzację całego środowiska uruchomieniowego przy użyciu Docker Compose (web, API, baza danych, backupy) z zabezpieczeniem produkcyjnego ruchu przez Cloudflare Tunnel oraz przygotowanie podstawowego zestawu testów automatycznych (jednostkowych i integracyjnych).

Poza zakresem MVP (*Minimum Viable Product*) pozostają m.in. natywne płatności online (PSP), zewnętrzny object storage i galeria albumów po imprezie (obecny upload lokalny JPG/PNG/WebP/PDF jest w zakresie), powiadomienia typu push (*push notifications*), publikacja aplikacji mobilnej w oficjalnych sklepach (App Store / Google Play), formalne testy obciążeniowe rzędu 10k rekordów przy 50 RPS, a także konfiguracja produkcyjnego certyfikatu TLS poza lokalnym zakresem testowym.

1.2 Wizja rozwiązania

RacePortal ma być **jednym miejscem** odkrywania i obsługi startów motorsportowych w Polsce: kierowca znajduje wydarzenie, sprawdza wymagania względem własnego garażu i składa zgłoszenie; organizator publikuje imprezę i zarządza listą uczestników; administrator moderuje treści i role.

Wizja produktowa zakłada:

- publiczny katalog z trzema widokami (lista, mapa, kalendarz) na wspólnych filtrach;
- ujednolicony proces zapisu ze statusami i powiadomieniami e-mail;
- wyraźny podział ról `USER` / `ORGANIZER` / `ADMIN` (gość przegląda katalog bez konieczności zakładania konta);
- mobilną ścieżkę discovery (Expo) z konwersją zapisu na wersję webową;
- uruchamialne środowisko demonstracyjne (Docker Compose) oraz wdrożenie produkcyjne realizowane za pośrednictwem Cloudflare Tunnel.

Szczegóły warstw technicznych (aplikacja webowa React/Vite, API REST oparte o Spring Boot, baza danych MySQL, aplikacja mobilna Expo oraz współdzielone typy API w pakiecie `packages/api-types`) opisano w dalszej części pracy oraz w rozdziałach implementacyjnych. Kod źródłowy projektu utrzymywany jest w dedykowanym repozytorium [1].

1.3 Struktura pracy

Niniejsza praca inżynierska została podzielona na logiczne i spójne rozdziały, które w sposób wyczerpujący przeprowadzają czytelnika przez proces koncepcyjny, analityczny, projektowy oraz implementacyjny systemu RacePortal:

- **Rozdział 1 (Informacje wstępne)** — przedstawia cel i zakres pracy, wizję rozwiązania opartą na architekturze MVP, a także strukturę całego dokumentu.
- **Rozdział 2 (Słownik pojęć)** — definiuje kluczowe terminy dziedzinowe i technologiczne wykorzystane w pracy, takie jak wydarzenie, katalog, garaż, zgłoszenie, mechanizm dopasowania pojazdu (`carMatch`), a także pojęcia związane z bezpieczeństwem i architekturą (JWT, RBAC, OSRM, MVP).
- **Rozdział 3 (Opis problemu)** — charakteryzuje specyfikę rynku amatorskiego i półprofesjonalnego motorsportu w Polsce, wskazuje na problemy organizacyjne i informacyjne po stronie kierowców oraz organizatorów, a także uzasadnia potrzebę stworzenia zintegrowanej platformy.
- **Rozdział 4 (Kontekst biznesowy)** — omawia segmentację użytkowników, mapę interesariuszy, propozycję wartości systemu, kanały komunikacji, model działania platformy oraz uwarunkowania prawne i etyczne.
- **Rozdział 5 (Analiza SWOT)** — analizuje mocne i słabe strony projektu oraz identyfikuje rynkowe szanse i zagrożenia związane z wdrożeniem systemu.
- **Rozdział 6 (Planowanie projektu)** — przedstawia cele, zakres, metodykę pracy, organizację zespołu, harmonogram, zastosowaną infrastrukturę oraz szczegółową analizę ryzyka projektowego.
- **Rozdział 7 (Analiza wymagań)** — zawiera źródła pozyskania wymagań, wymagania ogólne, dziedzinowe, funkcjonalne, niefunkcjonalne, bezpieczeństwa oraz interfejsu użytkownika, a także diagramy UML (w tym przypadki użycia).
- **Rozdział 8 (Projekt systemu)** — omawia założenia architektoniczne, architekturę logiczną i fizyczną, projekt relacyjnej bazy danych (model ERD), projekt interfejsu REST API, warstwy webowej (Figma Make), aplikacji mobilnej oraz mechanizmów bezpieczeństwa.
- **Rozdział 9 (Realizacja)** — przedstawia organizację prac implementacyjnych, podział systemu na komponenty (backend Spring Boot, frontend React/Vite, aplikacja mobilna Expo), integrację oraz konteneryzację środowiska za pomocą Docker Compose.
- **Rozdział 10 (Szczegóły implementacji backendu)** — szczegółowo opisuje strukturę projektu backendowego, model domenowy, mechanizmy uwierzytelniania i rejestracji (JWT, rate limiting), obsługę wydarzeń, zgłoszeń, garażu oraz integrację z bazą danych.

- **Rozdział 11 (Szczegóły implementacji frontendu webowego)** — omawia strukturę aplikacji webowej, kluczowe widoki (katalog, szczegóły, dashboard użytkownika, garaż, panele administratora i organizatora) oraz responsywność i doświadczenie użytkownika.
- **Rozdział 12 (Szczegóły implementacji aplikacji mobilnej)** — przedstawia założenia, architekturę oraz implementację mobilnej wizytówki systemu opartej na ekosystemie Expo.
- **Rozdział 13 (Bezpieczeństwo systemu)** — prezentuje założenia ochrony danych, klasyfikację informacji, uwierzytelnianie, kontrolę dostępu (RBAC), walidację wejścia oraz analizę zagrożeń i ryzyka.
- **Rozdział 14 (Testowanie i weryfikacja systemu)** — opisuje strategię testowania, środowisko testowe, testy jednostkowe, integracyjne, E2E (Playwright) oraz weryfikację mechanizmów bezpieczeństwa.
- **Rozdział 15 (Prezentacja systemu)** — ilustruje scenariusze użycia platformy z perspektywy kierowcy, organizatora oraz administratora.
- **Rozdział 16 (Nakład pracy)** — podsumowuje liczbę przepracowanych godzin, podział zadań w zespole oraz narzędzia wspierające zarządzanie pracą.
- **Rozdział 17 (Wkład własny)** — szczegółowo opisuje indywidualny wkład poszczególnych członków zespołu w realizację projektu.
- **Rozdział 18 (Podsumowanie)** — przedstawia największe wyzwania, adaptacje w trakcie prac, końcowe ograniczenia systemu (w tym wydajnościowe RPS) oraz perspektywy dalszego rozwoju platformy.

Rozdział 2

Słownik pojęć

Wydarzenie (event) Zorganizowany start / impreza motorsportowa (np. track day, drift, rally), opisana datą, lokalizacją, kategorią i wymaganiami.

Katalog Publiczny wykaz wydarzeń z widokami lista, mapa i kalendarz oraz filtrami.

Garaż Zbiór pojazdów przypisanych do konta kierowcy, wykorzystywanych przy zapisie na wydarzenie.

Zgłoszenie (registration) Wniosek o udział w wydarzeniu, posiadający status (np. PENDING, APPROVED, CANCELED).

carMatch Mechanizm dopasowania kategorii/klasy pojazdu do kategorii wydarzenia (aliasy nazw).

Organizator Użytkownik z rolą ORGANIZER, uprawniony do tworzenia i zarządzania własnymi wydarzeniami oraz zgłoszeniami.

Administrator Użytkownik z rolą ADMIN, moderujący wydarzenia, wnioski o rolę organizatora i role kont.

Wizytówka mobilna Wersja aplikacji Expo ograniczona do przeglądania katalogu; zapis odbywa się na webie (CTA).

JWT JSON Web Token — token sesji używany do uwierzytelniania żądań API.

RBAC Role-Based Access Control — kontrola dostępu oparta o role.

OSRM Open Source Routing Machine — publiczny serwis wyznaczania trasy wykorzystany jako domyślny provider.

MVP Minimum Viable Product — minimalny zakres funkcji przyjęty do odbioru projektu.

RPS Requests Per Second — metryka obciążenia (np. docelowy test 50 RPS przy 10k rekordach).

Rozdział 3

Opis problemu

3.1 Charakterystyka rynku wydarzeń motorsportowych

Amatorski i półprofesjonalny motorsport w Polsce przeżywa w ostatnich latach dynamiczny rozwój. Obejmuje on liczne, zróżnicowane formy aktywności, takie jak wydarzenia typu *track day*, zawody w drifcie, rajdy amatorskie (KJS, Super OES), wyścigi górskie czy zmagania *time attack*. Infrastruktura, od dużych obiektów pokroju Toru Poznań czy Silesia Ring, po mniejsze tory szkoleniowe i lokalne obiekty, pozwala na organizację imprez o różnym stopniu zaawansowania.

Mimo rosnącej popularności tej formy spędzania czasu, rynek ten charakteryzuje się dużym rozproszeniem organizacyjnym. Informacje o terminach, regulaminach oraz procedurach rejestracji są często rozrzucone pomiędzy indywidualnymi stronami internetowymi organizatorów, fanpage'ami w mediach społecznościowych (głównie na platformie Facebook) oraz tematycznymi forami dyskusyjnymi. Brakuje spójnego, scentralizowanego ekosystemu, który agregowałby tego typu aktywności w sposób ustandaryzowany, ułatwiając tym samym obieg informacji.

3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników

Z perspektywy organizatorów, zarządzanie mniejszymi i średnimi wydarzeniami motorsportowymi wiąże się z szeregiem wyzwań logistycznych. Większość podmiotów wykorzystuje do zbierania zgłoszeń podstawowe i niepołączone ze sobą narzędzia, takie jak formularze Google, arkusze kalkulacyjne czy wiadomości e-mail.

Ręczne przetwarzanie zgłoszeń jest nie tylko czasochłonne, ale i wysoce podatne na błędy ludzkie (np. zgubienie zgłoszenia, błędne przypisanie do klasy startowej). Poważnym problemem jest również weryfikacja techniczna uczestników — organizator musi ręcznie oceniać, czy dany pojazd spełnia wymagania techniczne dla danej kategorii wydarzenia (np. pojemność sil-

nika, napęd, klatka bezpieczeństwa). Dodatkowo, komunikacja z uczestnikami odnośnie statusu ich zgłoszenia (oczekujące, zatwierdzone, odrzucone) wymaga indywidualnej wysyłki wiadomości, co staje się trudne do skalowania przy rosnącej liczbie uczestników na listach startowych i rezerwowych.

3.3 Problemy informacyjne po stronie kierowców i organizatorów

Rozproszenie danych o imprezach dotkliwie uderza również w samych uczestników. Kierowcy mają ogromną trudność ze znalezieniem aktualnych wydarzeń startowych w jednym miejscu. Aby zaplanować sezon lub chociażby weekendowy wyjazd na tor, muszą oni na bieżąco monitorować wiele niezależnych źródeł.

Często proces poszukiwania informacji sprowadza się do przedzierania się przez nieczytelne posty w mediach społecznościowych, gdzie kluczowe dane, takie jak koszty wpisowego, harmonogram czy limit miejsc, są trudne do odnalezienia. Następnie, uczestnik staje przed problemem oceny, czy jego samochód w ogóle kwalifikuje się do wybranej imprezy. Brak centralnego kalendarza oraz ujednoliconego, przejrzystego interfejsu sprawia, że cały proces — od znalezienia wydarzenia, po udany start — jest niepotrzebnie skomplikowany, a komunikacja na linii organizator–kierowca jest chaotyczna i opóźniona.

3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal

Projekt platformy RacePortal jest bezpośrednią odpowiedzią na wyżej zidentyfikowane problemy. Jego głównym założeniem jest dostarczenie **centralnego katalogu wydarzeń** oraz zautomatyzowanego, **ujednoliconego procesu zapisów**, w którym obie strony (kierowca i organizator) operują na tym samym zbiorze i formacie danych.

Stworzenie dedykowanego rozwiązania pozwoli na:

- zebranie wszystkich typów wydarzeń motorsportowych w łatwej do przeszukiwania bazie z widokiem mapy, listy i kalendarza;
- wprowadzenie mechanizmu „wirtualnego garażu” dla uczestników, z poziomu którego ich pojazdy mogą być automatycznie zestawiane z wymogami wydarzeń (mechanizm *carMatch*);
- uporządkowanie ról w systemie (Gość, Użytkownik/Kierowca, Organizator, Administrator), co usprawni przepływ informacji, walidację zgłoszeń oraz moderację treści;
- zapewnienie użytkownikom dostępu poprzez ścieżkę webową (pełna funkcjonalność) oraz mobilną wizytówkę (ułatwione odkrywanie wydarzeń z poziomu smartfona).

3.5 Aspekty techniczne i organizacyjne problemu

Rozwiązanie zdefiniowanych problemów biznesowych pociąga za sobą konieczność sprostania konkretnym wyzwaniom technicznym. Głównym zadaniem jest zaprojektowanie wydajnego i spójnego modelu relacyjnej bazy danych, który sprawnie powiąże wydarzenia, użytkowników, pojazdy z ich parametrami oraz listę zgłoszeń i ich statusy.

Kluczowe zagadnienia techniczne podjęte w ramach projektu obejmują:

- **Bezpieczeństwo i autoryzację:** implementację bezstanowego uwierzytelniania opartego o tokeny JWT oraz ścisłej, opartej na rolach kontroli dostępu (RBAC) do poszczególnych zasobów API;
- **Walidację logiczną:** realizację mechanizmów sprawdzających wymagania wydarzenia względem pojazdu (moduł *carMatch*);
- **Niezawodną komunikację asynchroniczną:** wdrożenie systemu powiadomień e-mail dostarczającego uczestnikom zmiany statusów, również w środowiskach izolowanych.

Rozdział 4

Kontekst biznesowy

Niniejszy rozdział sytuuje RacePortal w kontekście produktowym i organizacyjnym: identyfikuje segmenty odbiorców, udziałowców oraz propozycję wartości, a następnie opisuje kanały dystrybucji, zasoby i działania kluczowe, model funkcjonowania platformy, realność wdrożenia oraz uwarunkowania prawno-etyczne. Punktem wyjścia są założenia Dokumentu Założeń Wstępnych (DZW), karty projektu, Business Model Canvas (BMC / materiały KMR) oraz scenariusze walidacji wywiadami, konfrontowane z faktycznym zakresem rozwiązania.

Należy zaznaczyć, że segmentacja w materiałach KMR została sformułowana jako *założenie koncepcyjne*, a nie jako wynik reprezentatywnych badań rynkowych. Dokument *Interview Validation* definiuje hipotezy, skrypt wywiadu oraz metryki sukcesu; nie zawiera wyników przeprowadzonych wywiadów, dlatego w dalszej części nie są one przytaczane jako dane empiryczne. Analogicznie, wskazany w materiałach koncepcyjnych przedział wiekowy 16–40 lat traktuje się wyłącznie jako hipotezę dotyczącą grupy docelowej, a nie jako potwierdzoną charakterystykę demograficzną rynku.

4.1 Segmentacja użytkowników

Z perspektywy biznesowej wyróżnia się trzy główne segmenty rynku, których potrzeby determinują wartość RacePortal. Segmentację rynkową należy odróżnić od ról dostępowych w systemie (kontrola dostępu oparta o role): role techniczne opisują uprawnienia w aplikacji, natomiast segmenty opisują grupy odbiorców i ich problemy.

Odbiorcy informacji (kibice i pasjonaci) poszukują aktualnych terminów, lokalizacji oraz podstawowych informacji o wydarzeniach motorsportowych. Dla tej grupy kluczowy jest niski próg wejścia — możliwość przeglądania katalogu bez rejestracji — oraz czytelne filtry i widoki (lista, mapa, kalendarz). Segment ten odpowiada w dużej mierze użytkownikowi anonimowemu (gościowi) w warstwie aplikacji.

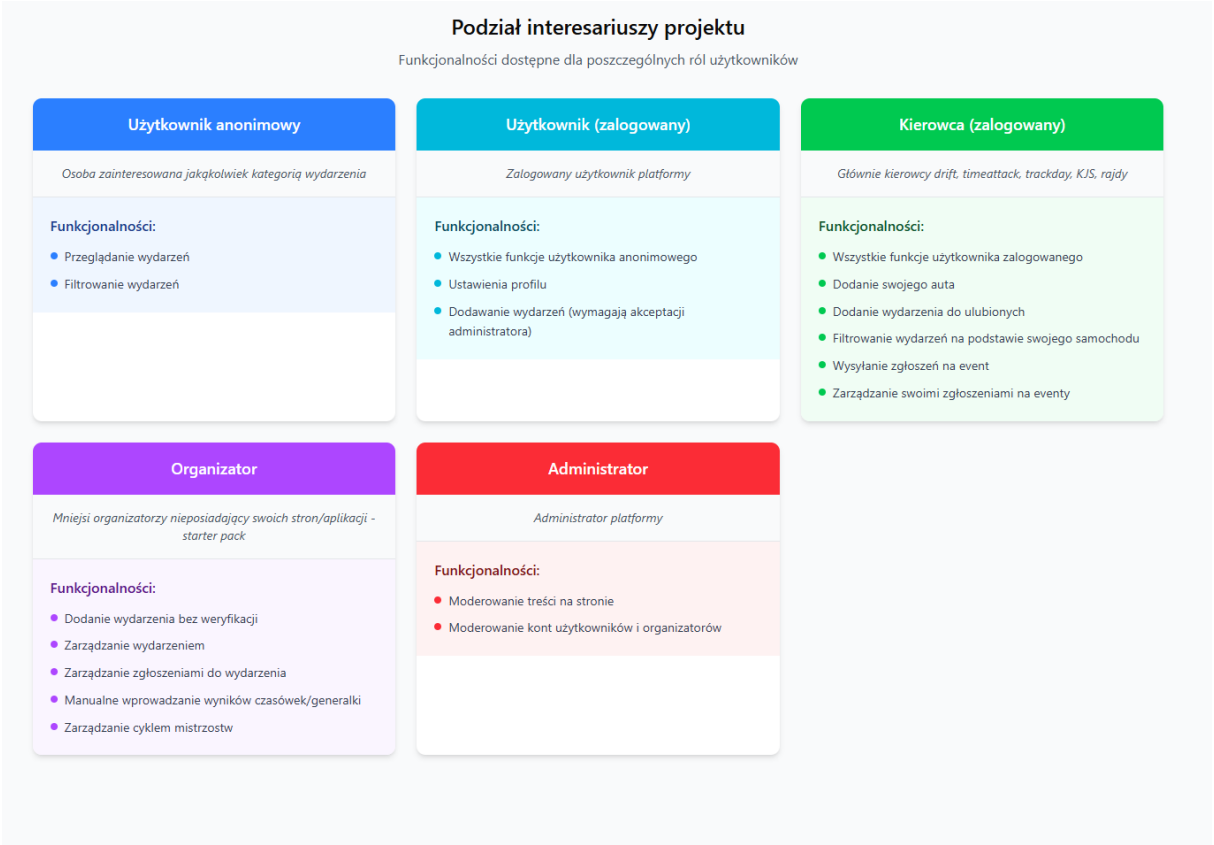
Uczestnicy wydarzeń (kierowcy) stanowią segment transakcyjny: potrzebują nie tylko znaleźć start, lecz także ocenić zgodność wymagań wydarzenia z posiadanym pojazdem, złożyć

zgłoszenie i śledzić jego status. W systemie segment ten jest obsługiwany przez konta użytkowników z dostępem do garażu i zgłoszeń. Hipoteza koncepcyjna KMR wiąże ten segment z rynkiem polskim i orientacyjnym przedziałem 16–40 lat; brakuje jednak wyników badań potwierdzających ten rozkład, dlatego nie jest on traktowany jako ustalenie empiryczne.

Organizatorzy wydarzeń dostarczają podaż treści i procesów zapisów. Ich potrzebą jest skuteczny kanał dotarcia do uczestników oraz narzędzie do publikacji imprezy i obsługi listy zgłoszeń (w tym komunikacji statusów). W praktyce produktowej dostęp organizatora wymaga weryfikacji wniosku przez operatora platformy, co stanowi mechanizm jakościowy, a nie osobny segment rynku.

Administrator nie jest segmentem rynkowym, lecz *rolą operacyjną* odpowiedzialną za funkcjonowanie platformy: moderację wydarzeń, rozstrzyganie wniosków o uprawnienia organizatora oraz nadzór nad jakością katalogu. W analizie biznesowej administrator należy do strony operatorskiej, a nie do grup klientów, do których kierowana jest propozycja wartości.

Wczesny, koncepcyjny podział funkcjonalności według poziomów dostępu (materiał projektowy zespołu) przedstawia rys. 4.1. Należy go interpretować jako mapę intencji produktowej; szczegółowe mapowanie na wymagania i projekt systemu opisano w rozdziałach 7–8.



Rysunek 4.1: Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny).

4.2 Interesariusze systemu

Analiza udziałowców obejmuje zarówno klientów platformy, jak i otoczenie opisane w dokumentacji SAI (m.in. zespół wytwórczy, media, właściciele obiektów, służby zabezpieczające imprezy). Poniżej przedstawiono karty kluczowych udziałowców w formacie środowiska `stakeholder` szablonu PJATK. W stosunku do wcześniejszej wersji dodano kartę gościa/kibica oraz właściciela toru / partnera obiektu, ponieważ obie grupy mają potrzeby odrębne od kierowcy i organizatora. Odwołania do wymagań wskazują rozdziały 7–8.

KARTA UDZIAŁOWCA	
Identyfikator:	U-00
Nazwa:	Gość / kibic (użytkownik anonimowy)
Opis:	Odbiorca informacji przeglądający publiczny katalog wydarzeń bez zakładania konta.
Typ udziałowca:	Użytkownik końcowy / odbiorca treści
Punkt widzenia:	Chce szybko znaleźć aktualne wydarzenia, lokalizacje i terminy bez bariery rejestracji.
Ograniczenia:	Bez konta nie może złożyć zgłoszenia ani prowadzić garażu; zależny od kompletności i aktualności katalogu publikowanego przez organizatorów.
Wymagania:	Publiczny katalog i filtry — rozdz. 7; widoki lista/mapa-/kalendarz — rozdz. 11.

Tabela 4.1: Karta udziałowca: Gość / kibic

KARTA UDZIAŁOWCA	
Identyfikator:	U-01
Nazwa:	Kierowca
Opis:	Użytkownik z kontem, który utrzymuje garaż pojazdów i składa zgłoszenia na wydarzenia motorsportowe.
Typ udziałowca:	Użytkownik końcowy / klient platformy
Punkt widzenia:	Chce szybko znaleźć starty, dopasować pojazd do wymagań wydarzenia i śledzić status zgłoszenia w jednym miejscu.
Ograniczenia:	Korzystanie z garażu i zapisów wymaga rejestracji oraz podania danych niezbędnych do obsługi zgłoszenia; skuteczność zapisu zależy od odpowiedzi organizatora i kompletności wymagań wydarzenia.
Wymagania:	Wymagania funkcjonalne katalogu, garażu i zgłoszeń — rozdz. 7; panel użytkownika — rozdz. 11.

Tabela 4.2: Karta udziałowca: Kierowca

KARTA UDZIAŁOWCA	
Identyfikator:	U-02
Nazwa:	Organizator
Opis:	Podmiot publikujący wydarzenia i zarządzający listą zgłoszeń uczestników.
Typ udziałowca:	Dostawca treści / partner biznesowy
Punkt widzenia:	Potrzebuje skutecznego kanału dotarcia do uczestników oraz narzędzia do zarządzania wydarzeniami, weryfikacji zgłoszeń i komunikacji statusów.
Ograniczenia:	Publikacja wymaga pozytywnej weryfikacji uprawnień; rozliczanie wpisowego odbywa się poza platformą, więc organizator nadal prowadzi proces płatności we własnym zakresie; wartość panelu zależy od liczby aktywnych kierowców.
Wymagania:	Panel organizatora i kontrola dostępu — rozdz. 7, 8; realizacja — rozdz. 10.

Tabela 4.3: Karta udziałowca: Organizator

KARTA UDZIAŁOWCA	
Identyfikator:	U-03
Nazwa:	Administrator (rola operacyjna)
Opis:	Operator platformy moderujący wydarzenia, role kont oraz wnioski o uprawnienia organizatora.
Typ udziałowca:	Właściciel / operator systemu
Punkt widzenia:	Zapewnia jakość katalogu, spójność uprawnień i bezpieczeństwo działania platformy.
Ograniczenia:	Skuteczność moderacji zależy od dostępności operatora i jakości zgłoszeń od użytkowników; przy wzroście skali rośnie nakład pracy na weryfikację treści i ról.
Wymagania:	Panel administracyjny oraz bezpieczeństwo dostępu — rozdz. 13; testy kontroli dostępu — rozdz. 14.

Tabela 4.4: Karta udziałowca: Administrator

4.3 Propozycja wartości systemu

Propozycja wartości RacePortal wynika z diagnozy rozproszenia informacji o wydarzeniach motorsportowych w Polsce. Dotychczasowe praktyki — grupy w mediach społecznościowych, witryny poszczególnych torów oraz statyczne kalendarze — nie zapewniają jednolitego procesu wyszukiwania imprez, porównania wymagań ani obsługi statusu zapisu. RacePortal odpowiada na ten deficyt jako **centralny katalog** oraz **warstwa operacyjna zapisów**, z narzędziami dopasowanymi do potrzeb kibiców, kierowców i organizatorów.

Dla kierowców wartość koncentruje się na: przejrzystym katalogu z widokami lista, mapa i kalendarz oraz wspólnymi filtrami; wirtualnym garażu pojazdów; wsparciu dopasowania pojazdu do wymagań wydarzenia; złożeniu zgłoszenia i otrzymywaniu informacji o statusie.

KARTA UDZIAŁOWCA	
Identyfikator:	U-04
Nazwa:	Właściciel toru / partner obiektu
Opis:	Podmiot dysponujący infrastrukturą (tor, obiekt) wykorzystywaną przy organizacji wydarzeń publikowanych na platformie.
Typ udziałowca:	Partner pośredni / interesariusz otoczenia
Punkt widzenia:	Jest zainteresowany widocznością obiektu oraz poprawnością informacji o lokalizacji wydarzeń odbywających się na jego terenie.
Ograniczenia:	Nie zarządza bezpośrednio zapisami uczestników w MVP; wpływ na treść katalogu jest pośredni (poprzez organizatorów lub kontakt z operatorem); nie posiada dedykowanego panelu partnerskiego.
Wymagania:	Katalog lokalizacji i dane wydarzeń — rozdz. 7, 8.

Tabela 4.5: Karta udziałowca: Właściciel toru / partner obiektu

Dla organizatorów wartość obejmuje publikację wydarzenia, zarządzanie listą zgłoszeń oraz komunikację statusów bez konieczności prowadzenia całego procesu wyłącznie w arkuszach i wiadomościach prywatnych. Dla odbiorców informacji wartość sprowadza się do bezpłatnego dostępu do terminów i lokalizacji, w tym do widoku mapy oraz wskazówek dojazdu.

Wyróżnikiem względem prostych kalendarzy branżowych jest integracja łańcucha *znalezienie wydarzenia → dopasowanie pojazdu → zgłoszenie → status*, a nie wyłącznie publikacja dat. Najistotniejszym ryzykiem biznesowym pozostaje odmowa migracji społeczności z grup społecznościowych do scentralizowanego narzędzia; bez aktualnej podaży wydarzeń katalog traci wartość dla kierowców (klasyczny problem platformy dwustronnej).

Tabela 4.6 przedstawia porównanie *modeli działania*, a nie empiryczne badanie konkurencji. Ilustruje, na czym polega zakładana przewaga RacePortal względem typowego sposobu organizacji informacji i zapisów w grupach społecznościowych.

Tabela 4.6: Porównanie modelu grup społecznościowych z modelem RacePortal (ujęcie koncepcyjne).

Kryterium	**Grupy społecznościowe**	**RacePortal (model docelowy)**
Wyszukiwanie wydarzeń	Zależne od postów i wyszukiwarki grupy	Ustrukturyzowane filtry i wspólny katalog
Porównanie wydarzeń	Ręczne, między rozproszonymi źródłami	Wspólny katalog z ujednoliconym opisem
Dopasowanie pojazdu	Weryfikacja po stronie uczestnika	Wsparcie dopasowania względem wymagań wydarzenia
Zgłoszenie udziału	Zależne od praktyki danego organizatora	Ujednolicony proces zgłoszenia
Status zgłoszenia	Różne kanały komunikacji	Centralny podgląd statusu i powiadomienia

4.4 Kanały komunikacji i dystrybucji

Głównym kanałem dostępu do produktu jest aplikacja webowa (`https://raceportal.pl`), umożliwiająca zarówno przeglądanie katalogu, jak i obsługę konta, garażu, zgłoszeń oraz paneli organizatora i administratora. Uzupełnieniem jest mobilna aplikacja-wizytówka, służąca do odkrywania wydarzeń i przekierowania zapisu do wersji webowej. Takie rozdzielenie obniża koszt wejścia informacyjnego, a jednocześnie koncentruje procesy wymagające konta w jednym miejscu.

Z perspektywy pozyskiwania użytkowników kluczowe są kanały organiczne i partnerskie, przewidziane także w materiałach BMC: obecność w mediach społecznościowych społeczności motorsportowej, bezpośrednie kontakty z klubami i organizatorami, promocja podczas imprez torowych oraz współpraca z właścicielami obiektów. W fazie MVP działania te mają charakter organizacyjny — ich celem jest zbudowanie podaży aktualnych wydarzeń i pierwszej grupy aktywnych kierowców, bez których katalog nie generuje efektu sieciowego.

Kanałem utrzymania relacji z użytkownikiem po rejestracji lub zgłoszeniu są powiadomienia e-mail (aktywacja konta, statusy zgłoszeń, istotne zmiany wydarzenia). Pełnią one funkcję komunikacyjną i operacyjną, a nie marketingową: informują o stanach procesu, zamiast prowadzić kampanie sprzedażowe. Szczegóły techniczne dostarczania poczty opisano w rozdziałach implementacyjnych.

4.5 Kluczowe zasoby i działania

Do zasobów kluczowych należą zarówno zasoby technologiczne, jak i biznesowo-operacyjne. Po stronie technologicznej są to: działająca aplikacja webowa i mobilna wizytówka, backend z trwałą bazą wydarzeń oraz konteneryzowane środowisko uruchomieniowe umożliwiające powtarzalne wdrożenie. Zasobem produktowym jest także katalog torów i treści wydarzeń — bez aktualnych danych platforma traci wartość niezależnie od jakości oprogramowania.

Po stronie biznesowej i operacyjnej kluczowi są: zespół projektowy utrzymujący rozwój i moderację; relacje z organizatorami i obiektami (źródło podaży wydarzeń); domena oraz niskokosztowa infrastruktura hostingowa; a także dokumentacja procesów i scenariusze prezentacji ułatwiające weryfikację produktu. W ujęciu BMC zasoby te są komplementarne: technologia bez relacji z organizatorami nie zapełni katalogu, a relacje bez sprawnego narzędzia nie utrzymają powtarzalnego procesu zapisów.

Działania kluczowe obejmują: rozwój funkcji zgodnych z MVP; pozyskiwanie i weryfikację organizatorów; moderację treści; utrzymanie dostępności serwisu; komunikację z użytkownikami (w tym powiadomienia procesowe); a także działania na rzecz budowy społeczności i zaufania do katalogu. Działania marketingowe pozostają konieczne dla przełamania ryzyka niskiego zainteresowania, lecz w MVP nie są zautomatyzowane w oprogramowaniu.

4.6 Model działania platformy

RacePortal funkcjonuje jako platforma dwustronna: organizatorzy dostarczają podaż wydarzeń, a kierowcy generują popyt na starty. Odbiorcy informacji korzystają z efektu sieciowego katalogu bez bariery rejestracji, co obniża koszt wejścia. Administrator — jako rola operacyjna — pełni funkcję gwaranta jakości poprzez akceptację wydarzeń i uprawnień.

W fazie MVP platforma **nie** rozlicza płatności za udział. Dla wydarzeń płatnych system obsługuje statusy zgłoszenia oraz dołączenie dowodu wpłaty, natomiast przelew lub inna forma rozliczenia odbywa się poza platformą, według zasad organizatora. Model darmowego dostępu obejmuje przeglądanie katalogu oraz korzystanie z narzędzi kierowcy i panelu organizatora. Na tym etapie weryfikacja wartości produktu opiera się na użyteczności operacyjnej, a nie na monetyzacji.

Z punktu widzenia utrzymania platformy materiały BMC wskazują potencjalne, *przyszłe* strumienie przychodów: płatną promocję / wyróżnienie wydarzeń, reklamy powiązane z motorsportem oraz — w dłuższym horyzoncie — prowizje od zapisów lub biletów. Równolegle występują koszty stałe i zmienne: hosting i domena, obsługa poczty, rozwój oprogramowania, moderacja treści oraz ewentualne działania promocyjne. W obecnym etapie projekt ma charakter dydaktyczno-inżynierski i opiera się na infrastrukturze niskokosztowej oraz pracy zespołu; nie przedstawia się tu prognoz finansowych liczbowych, ponieważ nie zebrano danych rynkowych wystarczających do wiarygodnej estymacji.

Praktycznym sposobem sprawdzenia zainteresowania odpłatnymi funkcjami byłoby — po okresie bezpłatnego użytkowania — przeprowadzenie rozmów lub prostych testów oferty z aktywnymi organizatorami (np. wyróżnienie wydarzenia, ekspozycja w katalogu), zgodnie z kierunkiem walidacji założonym w dokumencie *Interview Validation*. Dopiero potwierdzenie gotowości do zapłaty za konkretną korzyść uzasadniałoby przejście od modelu darmowego MVP do wariantu monetyzacji.

4.7 Realistyczność wdrożenia i dalsze perspektywy

Realistyczność techniczna wdrożenia została potwierdzona uruchomieniem środowiska lokalnego oraz publikacją działającej instancji produkcyjnej w modelu niskokosztowym (maszyna wirtualna w chmurze z udostępnieniem poprzez tunel HTTPS). Szczegóły architektury fizycznej zawarto w rozdz. 8.3; w niniejszym podrozdziale istotniejsza jest ocena realności *biznesowej* utrzymania produktu.

Kluczowym warunkiem sukcesu nie jest sama dostępność serwisu, lecz pozyskanie organizatorów publikujących aktualne wydarzenia oraz utrzymanie kompletności katalogu. Bez regularnej podaży treści platforma nie różni się istotnie od statycznego kalendarza i nie motywuje kierowców do rejestracji. Dlatego realistyczny scenariusz rozwoju zakłada: bezpośred-

nie kontakty z organizatorami i obiektami, prosty proces publikacji, moderację zapewniającą wiarygodność oraz mechanizmy powiadomień ograniczające ryzyko dezaktualizacji informacji u uczestników.

Założona w DZW skala pilotażowa (rzędu tysięcy użytkowników miesięcznie przy profilu głównie odczytowym) jest spójna z niszowym charakterem amatorskiego motorsportu w Polsce i z ograniczonym budżetem operacyjnym projektu. Formalny test obciążeniowy o parametrach docelowych nie został domknięty w MVP i pozostaje ograniczeniem odbiorczym wskazanym w karcie projektu.

Dalsze perspektywy obejmują m.in. integrację płatności online, rozbudowę powiadomień, publikację aplikacji mobilnej w sklepach, eksport list startowych oraz ewentualną monetyzację oznaczonych treści promocyjnych. Elementy te stanowią roadmapę produktową, a nie stan bieżącej realizacji.

4.8 Kontekst prawny i etyczny

System przetwarza dane osobowe niezbędne do obsługi konta i procesów biznesowych, w tym adres e-mail, dane profilu, informacje o pojazdach w garażu, zgłoszenia oraz dane wydarzeń wprowadzane przez organizatorów. Już na etapie rzeczywistego przetwarzania tych danych — także w MVP — obowiązują podstawowe wymogi ochrony danych osobowych (m.in. zasada minimalizacji, poinformowanie użytkownika, możliwość realizacji wybranych praw). W interfejsie udostępniono politykę prywatności oraz regulamin; rejestracja wymaga ich akceptacji. Przy tworzeniu wydarzenia organizator potwierdza przetwarzanie danych zgodnie z polityką prywatności wyłącznie w celu publikacji i obsługi zgłoszeń. Kanałem kontaktowym w sprawach ochrony danych jest adres `rodo@raceportal.pl`; użytkownik może zamknąć własne konto.

Należy rozróżnić *obowiązki już wynikające z przetwarzania danych* od *uszprawnień organizacyjnych* przewidzianych na późniejszy etap skalowania (np. bardziej rozbudowane procedury wewnętrzne, sformalizowana polityka retencji, ewentualna ocena skutków dla ochrony danych). Ocena skutków dla ochrony danych (DPIA) nie jest automatycznie wymagana dla każdego systemu — jej konieczność zależy od charakteru i ryzyka przetwarzania. W pracy nie rozstrzyga się, czy dla RacePortal DPIA jest obligatoryjna; wskazuje się jedynie, że przy wzroście skali i zakresu danych decyzja ta powinna zostać ponownie oceniona.

W warstwie odpowiedzialności za treści: organizator odpowiada za merytoryczną treść publikowanych wydarzeń w ramach regulaminu, jednak nie wyłącza to obowiązków operatora platformy w zakresie udostępniania narzędzia, moderacji oraz reagowania na zgłoszenia nadużyć. Model przyjęty w projekcie łączy weryfikację organizatorów i moderację wydarzeń przez administratora z zasadami etycznymi opisanymi w dokumentacji SAI (transparentność prezen-

tacji, unikanie mechanizmów uzależniających, przeciwdziałanie publikacji niebezpiecznych lub nielegalnych aktywności). Ostateczne decyzje moderacyjne pozostają po stronie człowieka.

Rozdział 5

Analiza SWOT

5.1 Mocne strony (Strengths)

- Pełny przepływ ról: gość, kierowca, organizator, administrator.
- Trzy widoki katalogu na wspólnych filtrach (lista / mapa / kalendarz).
- Konteneryzacja i seed kont demonstracyjnych ułatwiają prezentację.
- Testy automatyczne (API, web E2E, unit mobile / api-types).
- Wspólne typy API ograniczają dryf web $\leftrightarrow$ mobile.

5.2 Słabe strony (Weaknesses)

- Brak bramki płatności (PSP) oraz object storage / galerii albumów (upload lokalny obrazów i PDF jest dostępny).
- Mobilka bez logowania i garażu (wizytówka).
- Zależność od publicznych serwisów map/trasy (CARTO/OSRM).
- Częściowe rozjazdy względem wewnętrznej Specyfikacji Formularzy.
- Brak formalnego testu obciążenia 10k rekordów / 50 RPS.

5.3 Szanse (Opportunities)

- Rozszerzenie o PSP, waitlistę, watchlistę i eksport list startowych.
- Integracje z kalendarzami (iCal) i powiadomienia push.
- Budowa społeczności wokół torów i organizatorów regionalnych.

5.4 Zagrożenia (Threats)

- Konkurencja ze strony grup społecznościowych i dedykowanych systemów zapisów.
- Zmiany limitów / dostępności publicznych API map i routingu.
- Wymogi RODO i odpowiedzialność za dane osobowe przy skali produkcyjnej.

Rozdział 6

Planowanie projektu

6.1 Cele projektu

Dostarczenie działającego MVP katalogu i zapisów z rolami użytkowników, mapą, mailami, mobilną wizytówką oraz dokumentacją i testami umożliwiającymi odbiór / prezentację.

6.2 Zakres projektu

Zgodnie z listą funkcji MVP 1–15 (katalog, filtry, detal, admin, konto, garaż, zgłoszenia, organizator, baza, maile, mapa, trasa, archiwum, mobile) — szczegóły w dokumentacji projektowej `docs/MVP.md`.

6.3 Metodyka pracy

Iteracyjne dostarczanie funkcji w monorepo, z bieżącą dokumentacją zmian (`docs/changes.md`) oraz weryfikacją testami po istotnych zmianach.

6.4 Organizacja zespołu

Zespół pięcioosobowy pracuje we wspólnym repozytorium Git, z podziałem na warstwy web / API / mobile / dokumentacja / diagramy. Szczegółowy opis wkładu — rozdz. 17.

6.5 Harmonogram pracy

Prace obejmują m.in. prototyp Figma Make, scaffold UI, implementację backendu Spring, parity funkcji web, testy, uproszczenie mobile do wizytówki, wdrożenie Oracle + Cloudflare Tunnel oraz domykanie dokumentacji odbiorczej.

6.6 Infrastruktura i środowisko pracy

Docker Desktop, Node.js (web/mobile), JDK/Maven (API w kontenerze), Expo / iOS Simulator lub Android Emulator, Playwright, GitHub, Cloudflare Tunnel (`cloudflared`), Oracle Cloud VM.

6.7 Analiza ryzyka projektowego

6.7.1 Analiza zagrożeń

- Niespójność wymagań formularzy vs implementacja — mitigacja: rejestr luk (`spec-conformity.md`).
- Awaria zewnętrznego OSRM/CARTO — mitigacja: graceful fallback komunikatów, możliwość wymiany providera.
- Dryf typów web/mobile — mitigacja: `packages/api-types`.
- Opóźnienia wynikające z rozrostu zakresu — mitigacja: świadome odłożenie galerii/P-SP/store.

Rozdział 7

Analiza wymagań

7.1 Źródła i sposób pozyskania wymagań

Wymagania dla platformy RacePortal zostały zebrane i sformułowane w oparciu o wielotorowy proces analityczny, ukierunkowany na rozwiązanie realnych problemów rynku motorsportowego. Głównymi źródłami pozyskania wymagań były:

- **Dokument Założeń Wstępnych (DZW) oraz Specyfikacja Wymagań Systemowych (SWS)**, które zdefiniowały biznesowy cel platformy i zakres funkcjonalności dla wersji MVP.
- **Obserwacja polskiego środowiska wydarzeń motorsportowych**, charakteryzującego się dotychczas znacznym rozproszeniem informacji, brakiem spójnych standardów zapisów oraz chaosem komunikacyjnym w mediach społecznościowych.
- **Bezpośrednie rozmowy z kierowcami oraz organizatorami imprez**, które pozwoliły zidentyfikować kluczowe problemy operacyjne, takie jak uciążliwa i powtarzalna weryfikacja techniczna pojazdów czy trudności w efektywnym zarządzaniu listą startową.
- **Osobiste doświadczenia od strony aktywnego kierowcy**, zdobyte przez członka zespołu, Michała Gutowskiego, które dostarczyły bezcennej, praktycznej wiedzy na temat bolączek uczestników i oczekiwań wobec nowoczesnych systemów rejestracji na zawody.

7.2 Wymagania ogólne i dziedzinowe

KARTA WYMAGANIA			
Identyfikator:	W01	Priorytet:	Wysoki
Nazwa:	Zintegrowany ekosystem wydarzeń motorsportowych		
Opis:	System musi agregować dane o imprezach w jednym miejscu, udostępniając je użytkownikom w sposób przejrzysty poprzez klasyczną listę, czytelny kalendarz oraz interaktywną mapę.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	Brak		

Tabela 7.1: Zintegrowany ekosystem wydarzeń motorsportowych

KARTA WYMAGANIA			
Identyfikator:	W02	Priorytet:	Wysoki
Nazwa:	Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów		
Opis:	Platforma ma automatyzować proces zapisów, weryfikując systemowo czy pojazd zawodnika wybrany z jego wirtualnego garażu spełnia regulaminowe wymogi danej kategorii wydarzenia. Odciąża to organizatorów i niweluje błędy ludzkie.		
Udziałowiec:	Kierowca, Organizator		
Wymagania powiązane:	F04, F05		

Tabela 7.2: Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów

KARTA WYMAGANIA			
Identyfikator:	W03	Priorytet:	Wysoki
Nazwa:	Przejrzysty podział ról w systemie		
Opis:	System opiera się na wyraźnym podziale uprawnień na: użytkowników (szukających startów), organizatorów (zarządzających swoimi imprezami) oraz administratorów (dbających o jakość, wsparcie i bezpieczeństwo platformy).		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	SEC02		

Tabela 7.3: Przejrzysty podział ról w systemie

7.3 Wymagania funkcjonalne

KARTA WYMAGANIA			
Identyfikator:	F01	Priorytet:	Wysoki
Nazwa:	Publiczny katalog i inteligentne filtrowanie		
Opis:	Każdy użytkownik platformy ma dostęp do bazy wydarzeń z intuicyjną możliwością filtrowania ofert po kategorii zawodów, dacie oraz interesującym go województwie.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	W01		

Tabela 7.4: Publiczny katalog i inteligentne filtrowanie

KARTA WYMAGANIA			
Identyfikator:	F02	Priorytet:	Wysoki
Nazwa:	Szczegóły wydarzenia i wyznaczanie trasy		
Opis:	Platforma udostępnia kompleksowe informacje o imprezie, w tym zintegrowaną funkcję mapy ułatwiającą zawodnikom wyznaczenie najszybszej trasy dojazdu na docelowy obiekt sportowy.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	W01		

Tabela 7.5: Szczegóły wydarzenia i wyznaczanie trasy

KARTA WYMAGANIA			
Identyfikator:	F03	Priorytet:	Wysoki
Nazwa:	Zarządzanie profilem i kontem		
Opis:	Zapewnienie prostego procesu zakładania konta, logowania oraz edycji profilu, umożliwiającego kierowcom sprawne wprowadzanie niezbędnych danych kontaktowych i licencyjnych.		
Udziałowiec:	Kierowca, Organizator		
Wymagania powiązane:	Brak		

Tabela 7.6: Zarządzanie profilem i kontem

KARTA WYMAGANIA			
Identyfikator:	F04	Priorytet:	Wysoki
Nazwa:	Wirtualny Garaż Kierowcy		
Opis:	Cyfrowa ewidencja pojazdów przypisana do profilu. Pozwala na jednorazowe wprowadzenie danych technicznych auta, co drastycznie przyspiesza wielokrotne zapisy na różne zawody w przyszłości.		
Udziałowiec:	Kierowca		
Wymagania powiązane:	W02		

Tabela 7.7: Wirtualny Garaż Kierowcy

KARTA WYMAGANIA			
Identyfikator:	F05	Priorytet:	Wysoki
Nazwa:	Szybkie aplikowanie na wydarzenia		
Opis:	Kierowca może zgłosić chęć udziału w wybranych zawodach poprzez wskazanie odpowiedniego pojazdu z garażu. Poprawne zgłoszenie natychmiast trafia do organizatora ze statusem oczekującym na akceptację.		
Udziałowiec:	Kierowca		
Wymagania powiązane:	W02, F04		

Tabela 7.8: Szybkie aplikowanie na wydarzenia

KARTA WYMAGANIA			
Identyfikator:	F06	Priorytet:	Średni
Nazwa:	Skuteczny system powiadomień		
Opis:	Wdrożenie powiadomień systemowych wewnątrz aplikacji oraz wiadomości e-mail, które na bieżąco informują użytkowników o istotnych zmianach, takich jak akceptacja lub odrzucenie wniosku o start.		
Udziałowiec:	Zalogowany Użytkownik		
Wymagania powiązane:	Brak		

Tabela 7.9: Skuteczny system powiadomień

KARTA WYMAGANIA			
Identyfikator:	F07	Priorytet:	Wysoki
Nazwa:	Panel Zarządzania dla Organizatora		
Opis:	Dedykowane narzędzie biznesowe dla twórców imprez, pozwalające na łatwą publikację nowych wydarzeń oraz masowe zarządzanie listą zgłoszonych kierowców i ich opłatami.		
Udziałowiec:	Organizator		
Wymagania powiązane:	W03		

Tabela 7.10: Panel Zarządzania dla Organizatora

KARTA WYMAGANIA			
Identyfikator:	F08	Priorytet:	Wysoki
Nazwa:	Panel Administratora i wsparcie użytkowników		
Opis:	Centralne miejsce dowodzenia pozwalające na weryfikację publikowanych treści, nadawanie uprawnień nowym organizatorom oraz sprawną obsługę zgłoszeń i problemów napotkanych przez użytkowników.		
Udziałowiec:	Administrator		
Wymagania powiązane:	W03		

Tabela 7.11: Panel Administratora i wsparcie użytkowników

KARTA WYMAGANIA			
Identyfikator:	F09	Priorytet:	Średni
Nazwa:	Mobilna aplikacja wizytówkowa		
Opis:	Dodatkowy punkt styku dla użytkowników smartfonów, ułatwiający szybkie przeglądanie katalogu imprez w podróży z opcją bezproblemowego przekierowania do głównej platformy webowej w celu sfinalizowania zapisu.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	Brak		

Tabela 7.12: Mobilna aplikacja wizytówkowa

7.4 Wymagania niefunkcjonalne

KARTA WYMAGANIA			
Identyfikator:	NF01	Priorytet:	Wysoki
Nazwa:	Wydajność i responsywność platformy		
Opis:	System musi działać płynnie, gwarantując krótki czas ładowania katalogu i bezproblemową obsługę nawet przy dużym zainteresowaniu wydarzeniami i wzmożonym ruchem sieciowym.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.13: Wydajność i responsywność platformy

KARTA WYMAGANIA			
Identyfikator:	NF02	Priorytet:	Wysoki
Nazwa:	Dostępność wieloplatformowa (RWD)		
Opis:	Interfejs aplikacji musi automatycznie dostosowywać się do rozmiaru ekranu użytkownika, zapewniając pełen komfort korzystania na smartfonach, tabletach oraz komputerach stacjonarnych.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.14: Dostępność wieloplatformowa (RWD)

KARTA WYMAGANIA			
Identyfikator:	NF03	Priorytet:	Średni
Nazwa:	Niezawodność i łatwość wdrażania		
Opis:	Platforma powinna zapewniać wysoką bezawaryjność oraz pozwalać na łatwe instalowanie nowych funkcjonalności bez długotrwałych przerw w dostępności usług dla kierowców.		
Udziałowiec:	Administrator		
Wymagania powiązane:	Brak		

Tabela 7.15: Niezawodność i łatwość wdrażania

7.5 Wymagania bezpieczeństwa

KARTA WYMAGANIA			
Identyfikator:	SEC01	Priorytet:	Wysoki
Nazwa:	Bezpieczeństwo danych osobowych i kont		
Opis:	Zaawansowana ochrona kont użytkowników, bezpieczne uwierzytelnianie oraz rygorystyczne szyfrowanie haseł, gwarantujące pełną zgodność z wymogami ochrony prywatności (RODO).		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.16: Bezpieczeństwo danych osobowych i kont

KARTA WYMAGANIA			
Identyfikator:	SEC02	Priorytet:	Wysoki
Nazwa:	Ścisła kontrola dostępu do zasobów		
Opis:	Gwarancja systemowa, że kierowca widzi i zarządza wyłącznie własnymi pojazdami oraz zgłoszeniami, a organizator posiada uprawnienia do wglądu i edycji tylko swoich wydarzeń.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	W03		

Tabela 7.17: Ścisła kontrola dostępu do zasobów

KARTA WYMAGANIA			
Identyfikator:	SEC03	Priorytet:	Średni
Nazwa:	Odporność na nadużycia platformy		
Opis:	Implementacja mechanizmów chroniących platformę przed zautomatyzowanym spamem, atakami masowymi oraz próbami podszywania się pod oficjalną obsługę platformy w celu wyłudzenia informacji.		
Udziałowiec:	Administrator		
Wymagania powiązane:	Brak		

Tabela 7.18: Odporność na nadużycia platformy

7.6 Wymagania interfejsu użytkownika

KARTA WYMAGANIA			
Identyfikator:	UI01	Priorytet:	Wysoki
Nazwa:	Nowoczesny design branżowy		
Opis:	Atrakcyjny wizualnie, spójny interfejs oparty na ciemnym motywie z mocnymi, kontrastującymi akcentami, który naturalnie wpisuje się w dynamikę i estetykę branży motorsportowej.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.19: Nowoczesny design branżowy

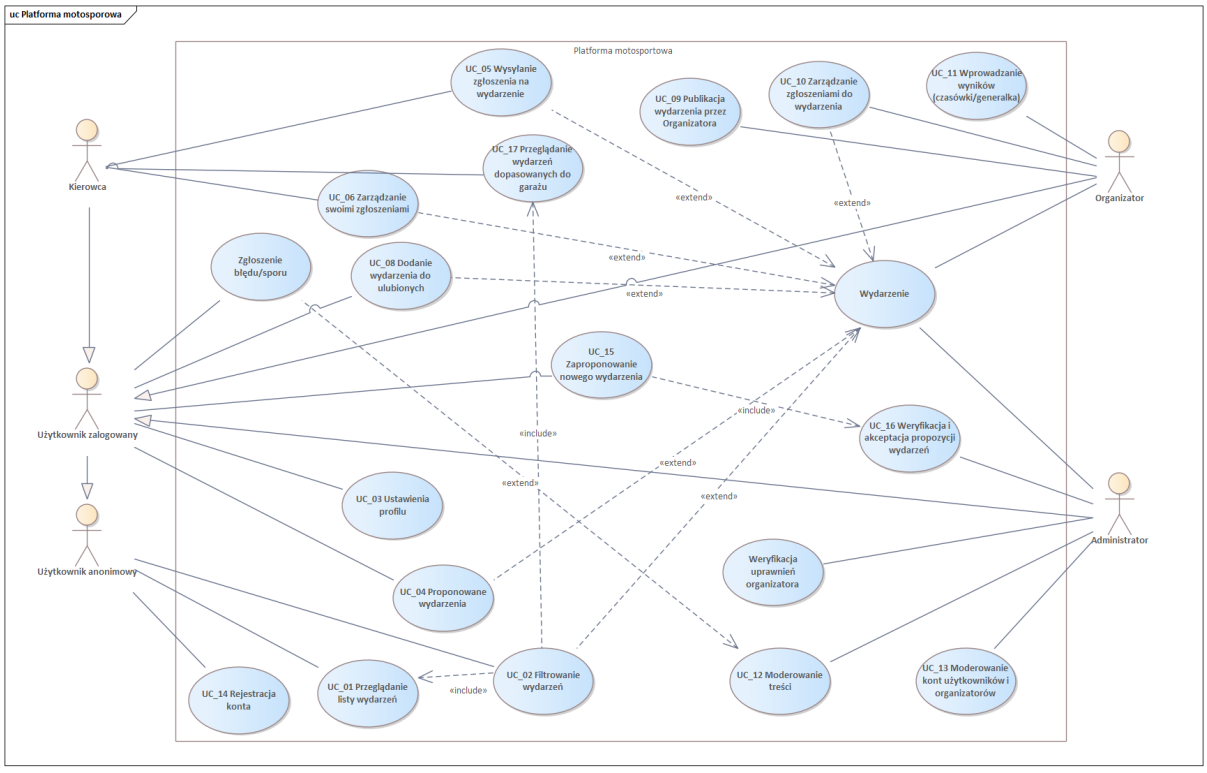
KARTA WYMAGANIA			
Identyfikator:	UI02	Priorytet:	Wysoki
Nazwa:	Intuicyjność i czytelność procesów		
Opis:	Kluczowy nacisk na jasne komunikaty dla użytkownika, widoczne wskaźniki ładowania oraz w pełni zrozumiałe formularze zapobiegające frustracji i pomyłkom podczas rejestracji na ważne zawody.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.20: Intuicyjność i czytelność procesów

7.7 Diagramy UML

7.7.1 Diagram przypadków użycia

Wczesny diagram przypadków użycia (materiały projektowe) przedstawia rys. 7.1.



Rysunek 7.1: Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna).

7.7.2 Diagram sekwencji: rejestracja i logowanie

7.7.3 Diagram sekwencji: zapis na wydarzenie

7.7.4 Diagram sekwencji: zarządzanie garażem

Uwaga: diagram klas / model danych przeniesiono do projektu systemu — podrozdział 8.4.1.

Rozdział 8

Projekt systemu

8.1 Założenia architektoniczne

Architektura trójwarstwowa: klienty (web, mobile) → API REST → MySQL. Statyczny frontend za nginx z proxy `/api`.

8.2 Architektura logiczna systemu

Moduły domenowe: użytkownicy/auth, wydarzenia, zgłoszenia, garaż, organizator, administracja, mapy/trasa, poczta.

8.3 Architektura fizyczna i wdrożeniowa

Kontenery lokalne: `web`, `api`, `mysql`, `backup`. Mobile poza Compose. Maile — SMTP Mailbux (poza Compose; `SMTP_PASSWORD` w `.env`).

Wdrożenie produkcyjne: maszyna Oracle Always Free (ARM) z Docker Compose; publiczny dostęp przez **Cloudflare Tunnel** (`cloudflared`) do nginx na porcie `:8081`. Domena `https://raceportal.pl` jest w tym samym koncie Cloudflare — TLS/HSTS i DNS po stronie Cloudflare, bez hostowania frontu na Cloudflare Pages.

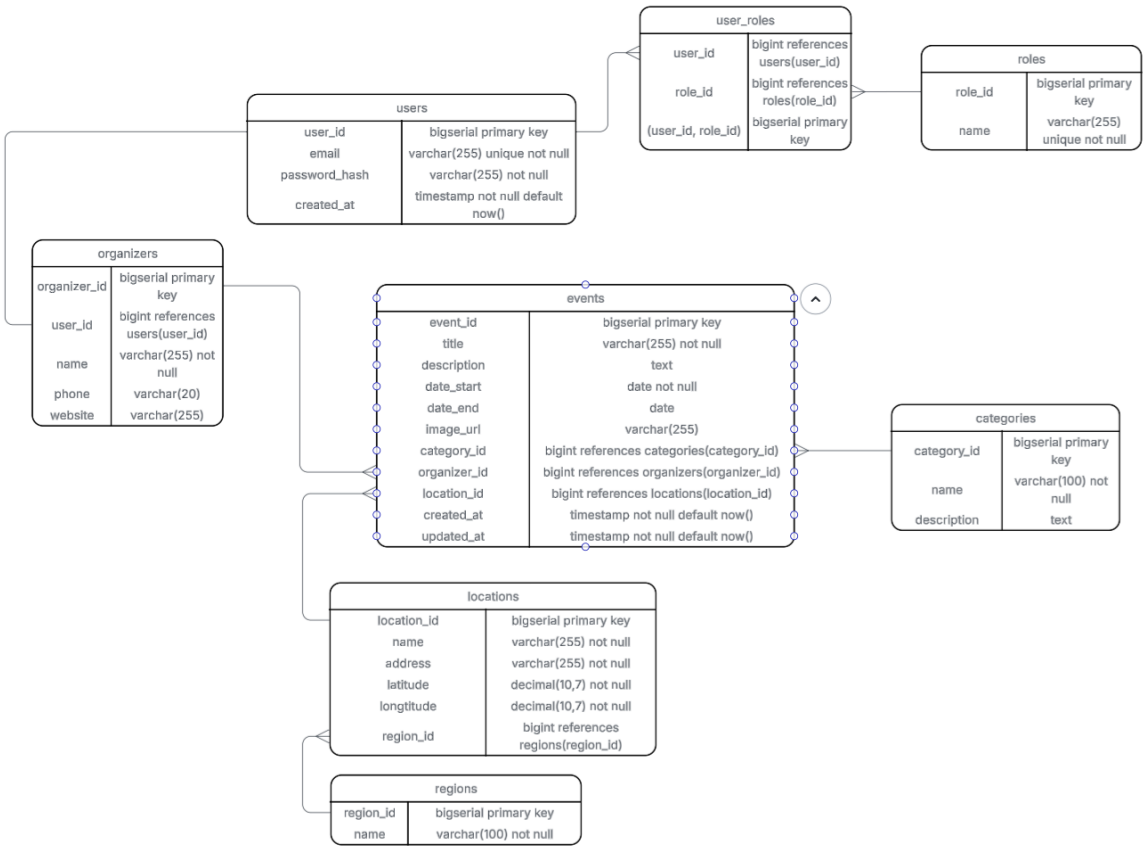
8.4 Projekt bazy danych

Encje m.in.: użytkownik, pojazd, wydarzenie, zgłoszenie, wniosek organizatora, **tor** (katalog PL seedowany przy starcie API, odczyt `GET /api/tracks`); statusy wydarzeń i zgłoszeń; współrzędne geograficzne. Wspólne presety torów dla UI: `TRACK_PRESETS` w `packages/api-types`.

8.4.1 Diagram klas

Rysunek 8.1 przedstawia wczesny model bazy danych (ERD) stanowiący punkt wyjścia projektu klas / tabel domenowych. W implementacji Spring Data JPA model został uproszczony

i dostosowany do MVP (m.in. role jako enum użytkownika, lokalizacja osadzona w wydarzeniu).



Rysunek 8.1: Diagram klas / model danych (ERD) — wersja koncepcyjna projektu.

8.5 Projekt interfejsu REST API

Interfejs API zaprojektowano w stylu REST nad protokołem HTTP, z wymianą danych w formacie JSON. Prefiks zasobów to `/api/...`. Kontrakt odpowiedzi jest wspólny dla klienta webowego i mobilnego; typy współdzielone utrzymywane są w pakiecie `packages/api-types`, co ogranicza rozjazd interpretacji pól pomiędzy warstwami.

Zasoby podzielono według odpowiedzialności domenowych. **Uwierzytelnianie i profil** (`/api/auth`) obejmują rejestrację, weryfikację e-mail, logowanie, odświeżanie i unieważnianie sesji, opcjonalne logowanie Google oraz operacje na profilu (`/me`). **Katalog wydarzeń** (`/api/events`) udostępnia listę z paginacją i filtrami, markery mapy, metadane kategorii, szczegóły wydarzenia oraz operacje tworzenia, edycji, anulowania i usuwania dla uprawnionych ról. Uzupełniają je zasoby klas startowych, listy startowej, wyników oraz komunikacji organizatora do uczestników konkretnego wydarzenia.

Zgłoszenia (`/api/registrations`) umożliwiają utworzenie zgłoszenia, podgląd własnych

zapisów, listę zgłoszeń wydarzenia, zmianę statusu, anulowanie, dołączenie dowodu płatności oraz przypisanie klasy. **Garaż** (`/api/garage`) obsługuje CRUD pojazdów użytkownika. **Organizator** (`/api/organizer`) obejmuje wniosek o rolę oraz listę zarządzanych wydarzeń, a **administrator** (`/api/admin`) — statystyki, użytkowników, moderację wydarzeń i rozstrzyganie wniosków organizatorskich.

Pozostałe zasoby wspierające to: katalog torów (`/api/tracks`), wyznaczanie trasy (`/api/maps/route`), upload plików (`/api/uploads`), powiadomienia in-app (`/api/notifications`), publiczne statystyki (`/api/stats`), zgłoszenia błędów (`/api/bug-reports`) oraz monitoring (`/api/health`).

Z punktu widzenia dostępu projekt rozróżnia zasoby publiczne (m.in. odczyt wydarzeń i torów, health, trasa, rejestracja/logowanie) oraz zasoby wymagające uwierzytelnienia i odpowiedniej roli. Błędy zwracane są w ujednoliconym formacie JSON (walidacja pól, kody HTTP 4xx/5xx), co upraszcza obsługę po stronie klientów. Szczegóły implementacji endpointów i przepływów opisano w rozdziale 10.

8.6 Projekt warstwy webowej

Routing React Router; AuthContext; strony katalogu, detalu, dashboardu, garażu, paneli; komponent mapy Leaflet.

8.6.1 Prototyp UI w Figma Make

Wyjściowym projektem interfejsu był plik Figma Make **Advanced UI/UX for RACEPORTAL** (ciemny motyw motorsportowy, akcent marki, Orbitron / typografia display, komponenty zbliżone do shadcn/ui). Prototyp obejmował m.in. stronę główną, katalog, logowanie/rejestrację, dashboard oraz szkice paneli organizatora i administratora.

Kod frontendu został przeniesiony z Figma Make do lokalnego monorepo (Vite/React), a następnie podpięty pod realne API (zamiast mocków / localStorage). Wczesna wizja produktu i ról — jeszcze przed pełną implementacją — jest też widoczna na diagramach koncepcyjnych (rys. 7.1, 4.1).

W toku realizacji UI dopracowano pod kątem responsywności, stanów pustych/błędów, PWA (manifest, service worker, dolna nawigacja mobilna) oraz spójności z tokenami marki (`-race-accent`).

8.7 Projekt aplikacji mobilnej

Stack Expo; nawigacja stack Eventy; publiczny klient HTTP; `EXPO_PUBLIC_API_URL` / `EXPO_PUBLIC_WEB_UR` CTA `Linking` do `/wydarzenia/:id`.

8.8 Projekt mechanizmów bezpieczeństwa

JWT w nagłówku Authorization (web); role na endpointach; walidacja DTO; CORS; CSP nginx.

Rozdział 9

Realizacja

9.1 Organizacja implementacji

Implementacja iteracyjna w monorepo, z dokumentowaniem zmian i utrzymywaniem środowiska Compose jako źródła prawdy uruchomieniowej.

9.2 Podział systemu na komponenty

9.2.1 Backend

Komponent backendowy stanowi warstwę serwerową RacePortal i odpowiada za udostępnienie API REST, trwałe przechowywanie danych oraz egzekwowanie reguł biznesowych i uprawnień. Został zrealizowany w technologii Spring Boot (wersja 3.3.5) na platformie Java 21, z relacyjną bazą MySQL oraz mapowaniem obiektowo-relacyjnym Spring Data JPA. Budowa i uruchamianie odbywają się w kontenerze (obraz Maven do kompilacji, obraz JRE do runtime), co zapewnia powtarzalność środowiska lokalnego i produkcyjnego.

Struktura kodu w pakiecie `pl.raceportal` odzwierciedla klasyczny podział warstwowy. Warstwa kontrolerów eksponuje zasoby HTTP (m.in. uwierzytelnianie, wydarzenia, zgłoszenia, garaż, tory, mapy, uploady, panele organizatora i administratora oraz health-check). Warstwa serwisów zawiera logikę dziedzinową — m.in. rejestrację i logowanie, zarządzanie wydarzeniami, przepływ statusów zgłoszeń, dopasowanie pojazdu do wymagań wydarzenia, powiadomienia e-mail, moderację oraz archiwizację. Warstwa repozytoriów zapewnia dostęp do encji domenowych (użytkownik, pojazd, wydarzenie, zgłoszenie, tor i encje powiązane). Obiekty DTO oraz walidacja Bean Validation oddzielają kontrakt API od modelu persystencji, a globalna obsługa wyjątków ujednolica odpowiedzi błędów.

Bezpieczeństwo dostępu oparto o Spring Security oraz tokeny JWT: część zasobów katalogu pozostaje publiczna (przeglądanie wydarzeń, torów, trasy), natomiast operacje konta, garażu, zgłoszeń i moderacji wymagają uwierzytelnienia oraz odpowiednich ról. Komponent backendowy realizuje również zadania wspierające demonstrację i utrzymanie systemu: inicjalizację

danych startowych (`DataInitializer`), katalog torów, cache wybranych list, a także cykliczną archiwizację minionych wydarzeń (`ArchiveService`, uruchamiana przy starcie aplikacji oraz codziennie według harmonogramu).

W podziale odpowiedzialności backend jest jedynym źródłem prawdy dla stanu domeny — klienci webowy i mobilny nie utrzymują własnej bazy wydarzeń ani zgłoszeń, lecz konsumują wspólny kontrakt JSON. Szczegółowy opis encji, endpointów i przepływów implementacyjnych przedstawiono w rozdziale 10.

9.2.2 Frontend webowy

Vite + React; integracja z API; stany ról i formularze dziedzinowe.

9.2.3 Aplikacja mobilna

Expo — katalog publiczny bez auth.

9.3 Integracja komponentów

Wspólny kontrakt JSON; typy w `api-types`; nginx reverse proxy `/api` w środowisku Compose.

9.4 Konteneryzacja i uruchamianie środowiska

`docker compose up -build -d`; weryfikacja `/api/health`; Expo: `cd mobile && npm start`. Publiczny dostęp produkcyjny: Cloudflare Tunnel → nginx `:8081`.

Rozdział 10

Szczegóły implementacji backendu

Rozdział niniejszy uszczegóławia realizację warstwy serwerowej RacePortal, której ogólny zarys przedstawiono w podrozdziale 9.2.1. Opis oparto o faktyczną strukturę pakietu `pl.raceportal` oraz zachowanie API REST.

10.1 Struktura projektu backendowego

Backend jest osobnym modułem Maven (`raceportal-backend`) w monorepozytorium. Rodzic projektu to `spring-boot-starter-parent` w wersji 3.3.5; język implementacji to Java 21. Do kluczowych zależności należą Spring Web, Spring Security, Spring Data JPA, Bean Validation, Spring Mail, sterownik MySQL, cache Caffeine oraz biblioteka JJWT. Opcjonalne logowanie Google wykorzystuje klienta Google API.

Kod aplikacji zorganizowano w pakietach odpowiadających odpowiedzialnościom:

- `controller` — ekspozycja zasobów HTTP;
- `service` — logika biznesowa i integracje (m.in. poczta, mapy, upload);
- `repository` — dostęp do danych przez Spring Data JPA;
- `domain` — encje i enumy statusów/ról;
- `dto` — kontrakty żądań i odpowiedzi z adnotacjami walidacji;
- `security` — JWT, ciasteczka sesji, limity prób, konfiguracja Spring Security;
- `web` — wyjątki domenowe i globalna obsługa błędów;
- `config`, `i18n` — konfiguracja serializacji oraz teksty wiadomości e-mail.

Klasa startowa `RacePortalApplication` włącza harmonogram zadań (`@EnableScheduling`), wykorzystywany m.in. do archiwizacji wydarzeń. Rodzic Maven oraz punkt wejścia aplikacji ilustrują listingi 10.1 i 10.2.

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.5</version>
</parent>
<properties>
  <java.version>21</java.version>
</properties>
```

Listing 10.1: Rodzic Maven i wersja Java backendu (`pom.xml`)

```java
@SpringBootApplication
@EnableScheduling
public class RacePortalApplication {
    public static void main(String[] args) {
        SpringApplication.run(RacePortalApplication.class, args);
    }
}
```

Listing 10.2: Klasa startowa Spring Boot z harmonogramem

10.2 Model domenowy i encje

Model persystencji odzwierciedla główne pojęcia katalogu i zapisów. Encja `User` reprezentuje konto: unikalny e-mail i nazwę użytkownika, skrót hasła, rolę (`USER`, `ORGANIZER`, `ADMIN`), dane profilu, flagi weryfikacji e-mail oraz pola związane z opcjonalnym kontem Google. Encja `Car` opisuje pojazd w garażu (marka, model, klasa, napęd, parametry techniczne, opony, wymagane dokumenty/wyposażenie). Encja `Event` przechowuje opis wydarzenia, lokalizację (w tym współrzędne), kategorię, status publikacji, warunki płatności oraz wymagania wobec uczestnika i pojazdu; wydarzenie jest powiązane z organizatorem (`User`).

Encja `Registration` modeluje zgłoszenie udziału: powiązanie użytkownika, wydarzenia i pojazdu, status, komentarze, dowód płatności oraz termin opłaty. Statusy zgłoszenia obejmują m.in. `PENDING`, `ACCEPTED`, `CONFIRMED` i `CANCELED`. Statusy wydarzenia obejmują m.in. `DRAFT`, `PENDING`, `APPROVED`, `REJECTED`, `ARCHIVED` i `CANCELLED`.

Dodatkowo w modelu występują: katalog torów (`Track`), wnioski o rolę organizatora (`OrganizerApplica`
odświeżające tokeny sesji (`RefreshToken`), klasy zawodów powiązane z wydarzeniem, wyniki, reguły punktacji, powiadomienia in-app oraz zgłoszenia błędów. Katalog torów nie jest kluczem obcym wydarzenia — lokalizacja wydarzenia jest przechowywana w polach wydarzenia, a katalog służy jako słownik wspomagający formularze i filtry. Enumy ról i statusów zgłoszeń (listingi 10.3–10.4) oraz fragment encji `User` (listing 10.5) stanowią rdzeń modelu RBAC i przepływu zapisów.

```java
public enum Role {
```

```
    USER,       // kierowca
    ORGANIZER,  // tworzy wydarzenia i zarzadza zgloszeniami
    ADMIN       // moderacja i administracja platformy
}
```

Listing 10.3: Role użytkowników (RBAC)

```
public enum RegistrationStatus {
    PENDING,    // czeka na decyzje organizatora
    ACCEPTED,   // przyjete (platne - oczekuje na dowod wplaty)
    CONFIRMED,  // potwierdzone miejsce
    CANCELED
}
```

Listing 10.4: Statusy zgłoszenia na wydarzenie

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @Column(nullable = false, unique = true, length = 190)
    private String email;

    @Column(name = "password_hash", nullable = false, length = 100)
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Role role = Role.USER;
    // ... profil, weryfikacja e-mail, tokenVersion
}
```

Listing 10.5: Kluczowe pola encji `User`

10.3 Rejestracja użytkowników i logowanie

Uwierzytelnianie zrealizowano w `AuthController` (`/api/auth`) oraz `AuthService`. Rejestracja kierowcy tworzy konto z rolą `USER` w stanie niezweryfikowanym i wysyła wiadomość aktywacyjną z linkiem `/verify-email/{token}` (czas ważności tokenu ograniczony). Do momentu weryfikacji system nie wydaje sesji użytkownika. Dostępna jest także ścieżka rejestracji organizatora, która równolegle zakłada wniosek o uprawnienia.

Logowanie hasłem weryfikuje poświadczenia i wystawia sesję: krótkotrwały token dostępu JWT oraz rotowany token odświeżający przechowywany w bazie w postaci skrótu. Token dostępu niesie m.in. identyfikator, e-mail, nazwę użytkownika, rolę oraz wersję tokenu

(tokenVersion), umożliwiając unieważnienie sesji przy wylogowaniu lub zmianie poświadczeń. Odświeżanie sesji odbywa się endpointem /refresh; wylogowanie unieważnia refresh i podnosi wersję tokenu. Opcjonalnie wspierane jest logowanie Google (weryfikacja idToken przy skonfigurowanym identyfikatorze klienta).

Ochronę przed nadużyciami zapewnia AttemptRateLimiter (m.in. limity prób logowania i weryfikacji). Konfiguracja Spring Security utrzymuje sesję bezstanową po stronie serwera i udostępnia publicznie m.in. endpointy rejestracji, logowania, weryfikacji, odświeżania oraz odczyt katalogu wydarzeń (listing 10.6). Wydanie sesji po udanym logowaniu lub weryfikacji e-mail realizuje issueSession (listing 10.7).

```
http
  .csrf(csrf -> csrf.disable())
  .sessionManagement(sm ->
      sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
  .authorizeHttpRequests(auth -> auth
      .requestMatchers(HttpMethod.POST,
          "/api/auth/login", "/api/auth/register",
          "/api/auth/verify-email", "/api/auth/refresh").permitAll()
      .requestMatchers(HttpMethod.GET, "/api/events/**").permitAll()
      .anyRequest().authenticated())
  .addFilterBefore(jwtAuthFilter,
      UsernamePasswordAuthenticationFilter.class);
```

Listing 10.6: Fragment SecurityFilterChain — sesja bezstanowa i ścieżki publiczne

```
private AuthResponse issueSession(User user) {
    String access = jwtService.generateToken(user);
    String refresh = refreshTokenService.issue(user);
    return new AuthResponse(access, refresh, toUserDto(user));
}
```

Listing 10.7: Wydanie sesji: JWT access + rotowany refresh

10.4 Zarządzanie profilami użytkowników

Profil zalogowanego użytkownika jest obsługiwany w ramach zasobów /api/auth (brak osobnego kontrolera profilu). Endpoint GET /me zwraca bieżące dane konta, a PATCH /me umożliwia aktualizację nazwy użytkownika, avatara, imienia i nazwiska, telefonu, adresu, odnośnika do Instagrama, deklaracji prawa jazdy kategorii B i licencji PZM oraz preferencji lokalizacji językowej i motywu interfejsu.

Zmiana hasła (POST /me/password) jest dostępna dla kont lokalnych i zablokowana dla kont utworzonych wyłącznie przez Google. Zmiana adresu e-mail przebiega dwuetapowo (żądanie + potwierdzenie), z powiadomieniem na dotychczasowy adres. Użytkownik może zamknąć własne konto po podaniu wymaganej frazy potwierdzającej. Takie podejście łączy zarządza-

nie tożsamością z kontem w jednym module autentycznym, upraszczając powierzchnię API (listing 10.8).

```
@GetMapping("/me")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserDto> me(
        @AuthenticationPrincipal UserPrincipal currentUser) {
    return ResponseEntity.ok(authService.me(currentUser.getId()));
}

@PatchMapping("/me")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserDto> updateMe(
        @AuthenticationPrincipal UserPrincipal currentUser,
        @Valid @RequestBody UpdateMeRequest request) {
    return ResponseEntity.ok(
        authService.updateMe(currentUser.getId(), request));
}
```

Listing 10.8: Endpointy profilu w `AuthController`

10.5 Obsługa wydarzeń

Zasób `/api/events` (`EventController`, `EventService`) realizuje publiczny katalog oraz zarządzanie wydarzeniami przez uprawnione role. Lista wydarzeń obsługuje paginację oraz filtry (m.in. tekst, kategoria, miasto, województwo, tor, zakres dat, status, płatność, dopasowanie do pojazdu z garażu). Publicznie domyślnie widoczne są wydarzenia zatwierdzone z datą bieżącą lub przyszłą. Osobny endpoint markerów dostarcza dane do widoku mapy.

Tworzenie i edycja wydarzeń wymagają roli organizatora lub administratora. Nowe wydarzenie organizatora trafia do statusu oczekującego na moderację (z powiadomieniem administratorów); administrator może publikować od razu jako zatwierdzone (listing 10.9). Edycja przez organizatora ponownie kieruje treść do moderacji i może uruchomić powiadomienia do osób już zgłoszonych. Anulowanie wydarzenia ustawia odpowiedni status, anuluje otwarte zgłoszenia i informuje uczestników. Uzupełnieniem cyklu życia jest `ArchiveService`, który przy starcie aplikacji oraz codziennie przenosi minione, zatwierdzone wydarzenia do archiwum.

```
@Transactional
@CacheEvict(cacheNames = "events", allEntries = true)
public EventResponse create(EventCreateRequest request,
                            UserPrincipal currentUser) {
    Event event = new Event();
    // ... mapowanie pol z requestu (nazwa, data, lokalizacja, oplaty)
    event.setStatus(currentUser.getRole() == Role.ADMIN
            ? EventStatus.APPROVED
            : EventStatus.PENDING);
    event.setOrganizer(organizer);
    event = eventRepository.save(event);
    if (event.getStatus() == EventStatus.PENDING) {
        notificationService.notifyAdminsPendingEvent(event);
```

```
    }
    return serialize(event);
}
```

Listing 10.9: Tworzenie wydarzenia — status zależny od roli

10.6 Obsługa zgłoszeń

Zgłoszenia obsługuje `RegistrationController` (`/api/registrations`) przy wymaganiu uwierzytelnienia. Utworzenie zgłoszenia wiąże kierowcę, zatwierdzone wydarzenie przyjmujące zapisy oraz — opcjonalnie — pojazd z garażu; system może przypisać klasę startową według reguł wydarzenia. Początkowy status to `PENDING`.

Zmiany statusów podlegają walidacji przejść zależnej od charakteru wydarzenia. Dla imprez płatnych typowy przebieg obejmuje akceptację (`ACCEPTED`) z wyznaczeniem terminu opłaty, dołączenie dowodu płatności przez kierowcę oraz potwierdzenie (`CONFIRMED`). Dla imprez bezpłatnych możliwe jest przejście z `PENDING` bezpośrednio do `CONFIRMED`. Kierowca może anulować własne zgłoszenie w dopuszczalnych stanach. Operacje listy zgłoszeń wydarzenia oraz zmiany statusów są ograniczone do organizatora-właściciela lub administratora. Kluczowe zmiany generują transakcyjne wiadomości e-mail (utworzenie, status, anulowanie). Maszynę stanów ilustruje listing 10.10.

```
private void validateTransition(RegistrationStatus from,
        RegistrationStatus to, Registration registration, Event event) {
    if (from == RegistrationStatus.CANCELED
            || from == RegistrationStatus.CONFIRMED) {
        throw ApiException.badRequest(
            "Nie mozna zmienic statusu zgloszenia zakonczonego");
    }
    boolean valid = switch (from) {
        case PENDING -> event.isPaid()
            ? (to == RegistrationStatus.ACCEPTED
                || to == RegistrationStatus.CANCELED)
            : (to == RegistrationStatus.CONFIRMED
                || to == RegistrationStatus.CANCELED);
        case ACCEPTED -> to == RegistrationStatus.CONFIRMED
                || to == RegistrationStatus.CANCELED;
        default -> false;
    };
    if (!valid) {
        throw ApiException.badRequest(
            "Nieprawidlowe przejscie statusu zgloszenia");
    }
    if (to == RegistrationStatus.CONFIRMED
            && from == RegistrationStatus.ACCEPTED
            && (registration.getPaymentProofUrl() == null
                || registration.getPaymentProofUrl().isBlank())) {
        throw ApiException.badRequest(
            "Brak potwierdzenia przelewu");
    }
```

```
29 }
```

Listing 10.10: Walidacja przejść statusu zgłoszenia

10.7 Obsługa garażu

Garaż udostępnia `GarageController` (`/api/garage`) wyłącznie użytkownikom zalogowanym. Serwis umożliwia listowanie, dodawanie, edycję i usuwanie pojazdów należących do bieżącego konta. Przy tworzeniu wymagane są co najmniej marka i model; pozostałe atrybuty techniczne i deklaracje (np. OC, przegląd, klatka) służą późniejszemu dopasowaniu do wymagań wydarzeń.

Ograniczenia spójności chronią otwarte procesy zapisów: przy aktywnym zgłoszeniu zablokowana jest zmiana kluczowych cech pojazdu, a usunięcie pojazdu nie jest możliwe, gdy bierze on udział w zgłoszeniach w stanach uniemożliwiających bezpieczne odpięcie (listing 10.11). Dzięki temu garaż pozostaje spójny z modułem zgłoszeń i regułami *carMatch*.

```
1  @Transactional
2  public void delete(String userId, String carId) {
3      Car car = carRepository.findByIdAndUser_Id(carId, userId)
4          .orElseThrow(() -> ApiException.notFound("Nie znaleziono auta"));
5
6      if (registrationRepository.existsByCar_IdAndStatusIn(
7              carId, OPEN_STATUSES)) {
8          throw ApiException.badRequest(
9              "Brak mozliwosci usuniecia auta ze wzgledu na otwarte zgloszenie");
10     }
11     registrationRepository.clearCarId(carId);
12     carRepository.delete(car);
13 }
```

Listing 10.11: Usuwanie pojazdu z blokadą przy otwartym zgłoszeniu

10.8 Panel administratora i organizatora

Panel administracyjny eksponuje `AdminController` pod prefiksem `/api/admin` z wymaganiem roli `ADMIN`. Udostępnia m.in. statystyki platformy, listę użytkowników, zmianę ról (z zabezpieczeniami przed usunięciem ostatniego administratora), kolejki wydarzeń do moderacji, zmianę statusów wydarzeń oraz rozstrzyganie wniosków o rolę organizatora (akceptacja skutkuje nadaniem roli i powiadomieniem e-mail).

Ścieżki organizatora obejmują `OrganizerController` (`/api/organizer`): podgląd własnego wniosku, złożenie wniosku o rolę oraz listę wydarzeń zarządzanych przez organizatora (administrator widzi pełny zbiór). Operacje tworzenia i edycji wydarzeń oraz zarządzania

zgłoszeniami uczestników są realizowane przez zasoby wydarzeń i zgłoszeń z kontrolą dostępu opartą o role i własność zasobu. Taki podział utrzymuje czytelne granice RBAC bez dublowania całej logiki w jednym kontrolerze. Klasę panelu administratora przedstawia listing 10.12.

```
@RestController
@RequestMapping("/api/admin")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    @GetMapping("/events/pending")
    public ResponseEntity<List<EventResponse>> pendingEvents() {
        return ResponseEntity.ok(adminService.pendingEvents());
    }

    @PatchMapping("/organizer-applications/{id}")
    public ResponseEntity<OrganizerApplicationResponse>
            updateOrganizerApplication(@PathVariable String id,
                @Valid @RequestBody
                OrganizerApplicationStatusUpdateRequest request) {
        return ResponseEntity.ok(
            adminService.updateOrganizerApplicationStatus(
                id, request.status()));
    }
}
```

Listing 10.12: Panel administratora — RBAC na poziomie klasy

10.9 Integracja z bazą danych

Trwałość danych zapewnia MySQL w połączeniu ze Spring Data JPA. Konfiguracja połączenia pochodzi ze zmiennych środowiskowych (w środowisku Compose hostem jest usługa kontenerowa bazy). Aplikacja działa z `spring.jpa.open-in-view=false`. Schemat jest utrzymywany trybem `ddl-auto` (domyślnie `update`); w projekcie nie zastosowano Flyway ani Liquibase — jest to świadome uproszczenie MVP, akceptowalne przy kontrolowanym środowisku demonstracyjnym, lecz wymagające ostrożności przy dalszym rozwoju produkcyjnym.

Dla list wydarzeń zastosowano cache Caffeine (krótki czas życia), invalidowany m.in. przy archiwizacji. Seed danych startowych (`DataInitializer`) zapewnia konta demonstracyjne, katalog torów oraz przykładowe treści ułatwiające prezentację ról. Kluczowe ustawienia JPA i cache pokazuje listing 10.13.

```
spring:
  datasource:
    url: ${DATABASE_URL:jdbc:mysql://localhost:3306/raceportal}
    username: ${DATABASE_USERNAME:raceportal}
    password: ${DATABASE_PASSWORD:raceportal}
  jpa:
    hibernate:
      ddl-auto: ${JPA_DDL_AUTO:update}
    open-in-view: false
```

```
  cache:
    type: caffeine
    cache-names: events
    caffeine:
      spec: maximumSize=500,expireAfterWrite=20s
```

Listing 10.13: Konfiguracja MySQL/JPA i cache Caffeine (`application.yml`)

10.10 Obsługa wyjątków i walidacja

Wejście API jest walidowane adnotacjami Bean Validation na rekordach DTO (`@Valid` w kontrolerach). Błędy reguł biznesowych i autoryzacji są reprezentowane przez `ApiException` z odpowiednim kodem HTTP (m.in. 400, 401, 403, 404, 409, 429).

`GlobalExceptionHandler` (`@RestControllerAdvice`) mapuje wyjątki na jednolite odpowiedzi JSON, w tym błędy walidacji pól, naruszenia integralności danych, odmowy dostępu oraz limity liczby prób (z nagłówkiem `Retry-After` przy 429). Dzięki temu klienci webowy i mobilny otrzymują przewidywalny kontrakt błędów, a szczegóły techniczne wyjątków nie są niekontrolowanie ujawniane użytkownikowi końcowemu. Fabryki `ApiException` oraz handler walidacji przedstawiają listingi 10.14 i 10.15.

```
public static ApiException badRequest(String message) {
    return new ApiException(HttpStatus.BAD_REQUEST, message);
}
public static ApiException unauthorized(String message) {
    return new ApiException(HttpStatus.UNAUTHORIZED, message);
}
public static ApiException forbidden(String message) {
    return new ApiException(HttpStatus.FORBIDDEN, message);
}
public static ApiException notFound(String message) {
    return new ApiException(HttpStatus.NOT_FOUND, message);
}
public static ApiException conflict(String message) {
    return new ApiException(HttpStatus.CONFLICT, message);
}
public static ApiException tooManyRequests(String message) {
    return new ApiException(HttpStatus.TOO_MANY_REQUESTS, message);
}
```

Listing 10.14: Fabryki wyjątków biznesowych API

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<ErrorResponse> handleApiException(ApiException ex) {
        return ResponseEntity.status(ex.getStatus())
            .body(new ErrorResponse(ex.getMessage(), ex.getDetails()));
    }
```

```
9
10      @ExceptionHandler(MethodArgumentNotValidException.class)
11      public ResponseEntity<ErrorResponse> handleValidation(
12              MethodArgumentNotValidException ex) {
13          Map<String, String> fieldErrors = new LinkedHashMap<>();
14          ex.getBindingResult().getFieldErrors().forEach(fe ->
15              fieldErrors.put(fe.getField(), fe.getDefaultMessage()));
16          return ResponseEntity.status(HttpStatus.BAD_REQUEST)
17              .body(new ErrorResponse("Nieprawidlowe dane", fieldErrors));
18      }
19  }
```

Listing 10.15: Globalna obsługa `ApiException` i błędów `@Valid`

Rozdział 11

Szczegóły implementacji frontendu webowego

11.1 Struktura aplikacji webowej

Katalog `web/src/app`: `pages/`, `components/`, `context/`, `lib/`.

11.2 Strona główna i katalog wydarzeń

Widoki Lista / Mapa / Kalendarz; debounce filtrów; „Wyczyść filtry”; paginacja listy vs markery overview.

11.3 Widok szczegółów wydarzenia

Opis, wymagania, zapis z wyborem auta, `carMatch`, mapa i trasa.

11.4 Logowanie i rejestracja

Formularze auth; weryfikacja kodu; reset hasła; opcjonalny Google Sign-In.

11.5 Dashboard użytkownika

Zgłoszenia użytkownika i szybkie informacje o nadchodzących startach.

11.6 Widok garażu

Formularz pojazdu, walidacja pól wymaganych, edycja i usuwanie (z ograniczeniami przy otwartym zgłoszeniu).

11.7 Panel organizatora

Tworzenie/edycja wydarzeń, lista zgłoszeń, zmiana statusów.

11.8 Panel administratora

Moderacja wydarzeń, wnioski organizatora (zaakceptuj / odrzuć → rola `ORGANIZER`), zarządzanie rolami, zgłoszenia błędów. Formularz wniosku użytkownika: `/zostan-organizatorem`.

11.9 Mapa i elementy lokalizacyjne

Leaflet + kafelki CARTO; pickery lokalizacji; proxy trasy `/api/maps/route`.

11.10 Responsywność i doświadczenie użytkownika

Układ dostosowany do desktop/mobile web; motyw z akcentem `-race-accent`.

Rozdział 12

Szczegóły implementacji aplikacji mobilnej

12.1 Założenia aplikacji mobilnej

Mobilka ma wspierać odkrywanie wydarzeń, a nie dublować pełnego klienta webowego.

12.2 Architektura rozwiązania mobilnego

Expo SDK, React Navigation (stack Eventy), ThemeProvider, cienki klient API bez JWT.

12.3 Lista wydarzeń i widok szczegółów

Filtry publiczne; przełączanie Lista / Kalendarz / Mapa; detal z metadanymi wydarzenia.

12.4 Nawigacja i interfejs użytkownika

Pojedynczy stack: `EventsList` → `EventDetail`; dark UI spójny z marką.

12.5 Integracja z API

Publiczne GET (events, markers, health); konfiguracja hosta API (localhost / LAN / Android emulator).

12.6 Przekierowanie do wersji webowej

Przycisk „Zapisz się na stronie” — `Linking.openURL(webEventUrl(id))`.

12.7 Ograniczenia wersji mobilnej

Brak auth, garażu, paneli ról i archiwum w aplikacji; zależność od dostępności weba przy konwersji zapisu.

Rozdział 13

Bezpieczeństwo systemu

13.1 Założenia bezpieczeństwa

Ochrona kont i danych zgłoszeń na poziomie ASVS L1 adekwatnym do MVP demonstracyjnego.

13.2 Klasyfikacja danych

Dane konta i profilu; dane pojazdów; dane zgłoszeń; treści wydarzeń (częściowo publiczne).

13.3 Uwierzytelnianie użytkowników

Hasło + JWT; opcjonalnie Google; brak sesji serwerowej klasycznej.

13.4 Autoryzacja i kontrola dostępu RBAC

Role USER / ORGANIZER / ADMIN egzekwowane w API i bramkach UI.

13.5 Wykorzystanie tokenów JWT

Token przechowywany po stronie klienta web; dołączany do żądań chronionych.

13.6 Walidacja danych wejściowych

Walidacja DTO po stronie API; walidacja formularzy UI; komunikaty pól `details`.

13.7 Ochrona zasobów użytkowników

Dostęp do garażu i zgłoszeń wyłącznie właściciela (oraz uprawnionych ról w panelach).

13.8 Bezpieczeństwo komunikacji

W dev HTTP lokalne; w prod TLS terminowany na Cloudflare Tunnel przed nginx; CSP / X-Frame-Options na nginx.

13.9 Analiza zagrożeń

Uwierzytelnianie, IDOR, XSS, CSRF w kontekście SPA, brute-force logowania (rate limit).

13.10 Analiza ryzyka

Ryzyko akceptowalne dla środowiska demo; produkcja wymaga twardego sekretu JWT, HTTPS i polityki haseł zsynchronizowanej backend↔UI.

13.11 Ograniczenia przyjętych zabezpieczeń

Brak 2FA; Google OAuth opcjonalny; brak WAF; publiczny OSRM bez SLA.

Rozdział 14

Testowanie i weryfikacja systemu

14.1 Strategia testowania

Piramida: unit (api-types, mobile client) → integracyjne API (MockMvc) → E2E Playwright (web + preview mobile).

14.2 Środowisko testowe

Docker Compose + opcjonalnie Expo web `:8082`; dokumentacja `docs/testy/TESTY.md`.

14.3 Testy jednostkowe

Vitest: `packages/api-types`, `mobile/tests`.

14.4 Testy integracyjne backendu

JUnit / MockMvc — scenariusze auth, eventy, garaż, zgłoszenia, RBAC, trasa.

14.5 Testy End-to-End aplikacji webowej

Playwright: ścieżki katalogu, auth, garaż, role (viewport desktop/mobile).

14.6 Testy aplikacji mobilnej

Unit URL API/WEB; E2E gość (lista, widoki, detal, CTA) na Expo web.

14.7 Testy mechanizmów bezpieczeństwa

Scenariusze braku dostępu do paneli obcych ról; walidacja wejścia.

14.8 Automatyzacja uruchamiania testów

Skrypty npm / `scripts/`; raporty w `docs/testy/wyniki/`.

14.9 Analiza wyników testów

Werdykty przebiegów odnotowywane w dokumentacji wyników; regresie łapane przed prezentacją.

14.10 Ograniczenia procesu testowania

Brak pełnych testów natywnych (Detox/Maestro); **brak formalnego load testu 10k rekordów / 50 RPS** w MVP — ograniczenie omówione w podsumowaniu.

Rozdział 15

Prezentacja systemu

15.1 Prezentacja głównych funkcjonalności

Demonstracja katalogu (3 widoki), zapisu kierowcy z `carMatch`, paneli org/admin oraz mobile CTA — zgodnie z `docs/scenariusz-prezentacji.md`.

15.2 Scenariusz użycia z perspektywy kierowcy

Login seed → garaż → filtr/katalog → zapis → dashboard statusów.

15.3 Scenariusz użycia z perspektywy organizatora

Login org → wydarzenie (presety torów) → zgłoszenia → zmiana statusu → podgląd maila na skrzynce (Mailbux).

15.4 Scenariusz użycia z perspektywy administratora

Login admin → moderacja wydarzenia / wniosku organizatora / ról.

15.5 Podsumowanie działania systemu

System realizuje spójny obieg informacji o wydarzeniach i zapisach w środowisku uruchamialnym lokalnie oraz pod adresem produkcyjnym za Cloudflare Tunnel.

Rozdział 16

Nakład pracy

16.1 Liczba godzin przeznaczona na realizację projektu

16.2 Podział zadań w zespole

Skrót obszarów — rozdz. 17. Szczegółowa tabela godzin: TODO.

16.3 Narzędzia wspierające zarządzanie pracą

Git/GitHub, dokumentacja Markdown w `docs/`, Docker, Cursor/IDE, Playwright, Figma Make, Cloudflare Dashboard.

16.4 Problemy organizacyjne i ich rozwiązanie

M.in. rozrost zakresu mobile — decyzja o wizytówce; luki Spec — rejestr `spec-conformity.md`; stabilizacja testów względem seedu/archiwizacji; uproszczenie hostingu (Oracle + Tunnel zamiast GCP Pages).

Rozdział 17

Wkład własny

Rozdział opisuje, kto odpowiadał za które obszary prac projektowych i implementacyjnych (imię i nazwisko). Podział odzwierciedla faktyczny udział zespołu; największy wkład merytoryczny i projektowy w warstwie produktu/UI wniosła Natalia Otrombke.

17.1 Natalia Otrombke

- projekt UI/UX w Figma Make (**Advanced UI/UX for RACEPORTAL**) — ekrany katalogu, auth, dashboard, paneli;
- przeniesienie i dopracowanie frontendu React (motyw, layout, spójność marki, kluczowe ścieżki UX);
- udział w definicji flow użytkownika (gość → kierowca → organizator) oraz w materiałach koncepcyjnych ról.

17.2 Oliwier Kasprowicz

- prace analityczne i dokumentacyjne (wymagania, prezentacje zespołowe);
- wsparcie przy diagramach i scenariuszach użycia;
- udział w przeglądach zakresu MVP i odbiorze funkcjonalności.

17.3 Michał Gutowski

- diagramy i materiały projektowe (m.in. rich picture, przepływy);
- wsparcie dokumentacji i testów scenariuszy prezentacji;
- udział w integracji i weryfikacji ścieżek użytkownika.

17.4 Miłosz Parkitny

- modelowanie i diagramy (m.in. przypadki użycia, podział funkcjonalności, model danych);
- wsparcie analizy interesariuszy i harmonogramów;
- udział w pracach implementacyjnych / poprawkach w repozytorium.

17.5 Wojciech Wronisz

- backend Spring Boot (API, JWT/RBAC, encje, seed, maile);
- Docker Compose, wdrożenie produkcyjne Oracle + **Cloudflare Tunnel**, Mailbux, PWA;
- dokumentacja uruchomienia i odbioru (`docs/`), testy API / E2E, integracja web $\leftrightarrow$ API.

Rozdział 18

Podsumowanie

18.1 Największe wyzwania i sposób ich rozwiązania

Spójność domeny zapisów i garażu, RBAC, mapa/trasa oraz utrzymanie monorepo web+API+mobile. Po stronie infrastruktury — uproszczenie hostingu do jednej VM z tunelowaniem Cloudflare zamiast rozdzielonego stacku Pages/Worker.

18.2 Zmiany i adaptacje w trakcie realizacji

Przejście mobile z parity na wizytówkę; doprecyzowanie formularzy; wspólny pakiet typów; migracja z eksperymentów GCP/Pages na Oracle + Cloudflare Tunnel.

18.3 Ograniczenia końcowej wersji systemu

Końcowa wersja RacePortal stanowi domknięte MVP demonstracyjne: realizuje centralny katalog wydarzeń, proces zapisów, wirtualny garaż, panele organizatora i administratora oraz mobilną wizytówkę katalogu. Jednocześnie celowo pozostawia poza zakresem funkcje, które wymagałyby osobnego nakładu produktowego, prawnego lub infrastrukturalnego. Ograniczenia te są spójne z kartą projektu oraz z opisem zakresu w podrozdziale 1.1; poniżej zestawiono je według obszarów.

Płatności i rozliczenia. System nie integruje natywnej bramki płatności online (PSP). Statusy opłat po stronie organizatora mają charakter operacyjny (oznaczenie w panelu), a nie księgowy — nie ma automatycznego pobierania składek startowych ani zwrotów. Decyzja wynika z kosztów onboardingu PSP, wymogów KYC/AML oraz ryzyka prawnego nieproporcjonalnego do celów akademickiego MVP.

Upload plików — stan obecny i pozostałe ograniczenia. W odróżnieniu od wczesnych założeń MVP (wyłącznie referencje URL) aplikacja webowa i API obsługują już lokalny upload

plików. Endpointy `POST /api/uploads/image` oraz `POST /api/uploads/pdf` (wymagają uwierzytelnienia) przyjmują multipart z limitem ok. 5 MB; typ pliku weryfikowany jest po sygnaturze bajtowej (JPG/PNG/WebP oraz PDF), a nie wyłącznie po nagłówku przeglądarki. Zapis trafia do katalogu na serwerze aplikacji i jest serwowany pod `GET /api/uploads/{uuid}.{ext}`. Funkcja jest wykorzystana m.in. przy:

- zdjęciach pojazdów w garażu,
- plakacie / grafice wydarzenia,
- załącznikach regulaminów PDF wydarzenia (`regulationsUrls`),
- opcjonalnym potwierdzeniu przelewu przy zgłoszeniu (`paymentProofUrl`).

Nadal dopuszczalne jest wklejenie zewnętrznego adresu http(s) jako alternatywy dla pliku. Poza zakresem pozostają: zewnętrzny object storage (S3/MinIO), skanowanie antywirusowe, polityki retencji i podpisane URL-e z ograniczonym dostępem, pełna galeria albumów po imprezie oraz upload zdjęć z klienta mobilnego (wizytówka Expo nie udostępnia formularzy create/edit z plikami).

Powiadomienia. Kanał e-mail (SMTP Mailbux) pokrywa potwierdzenia rejestracji, weryfikację konta oraz zmiany statusów zgłoszeń. Brakuje natywnych powiadomień push na urządzenia mobilne oraz rozbudowanych kampanii marketingowych. Powiadomienia in-app w aplikacji webowej stanowią uzupełnienie, a nie pełny system komunikacji wielokanałowej.

Aplikacja mobilna. Klient Expo pełni rolę świadomej wizytówki: publiczny przegląd katalogu (lista / mapa / kalendarz) z CTA do serwisu webowego. Nie obejmuje logowania, garażu, zgłoszeń ani paneli ról; aplikacja nie została opublikowana w App Store ani Google Play. Pełna parity z webem została świadomie odłożona, aby nie rozpraszać nakładu pracy przy domykaniu ścieżki zapisu na desktopie i PWA.

Integracje zewnętrzne i dane mapowe. Wyznaczanie trasy opiera się domyślnie na publicznym OSRM (z opcją Google Maps API przy kluczu). Zależność od zewnętrznych serwisów map oznacza możliwe limity dostępności i brak gwarancji SLA poza kontrolą zespołu. Katalog torów jest seedowany i rozszerzalny, lecz nie stanowi kompletnej, oficjalnej bazy wszystkich obiektów w Polsce.

Bezpieczeństwo i hardening produkcyjny. Przyjęto poziom zabezpieczeń adekwatny do MVP (JWT w ciasteczkach HTTP-only, RBAC, rate limiting, walidacja wejścia, TLS/HSTS terminowane na Cloudflare Tunnel). Poza zakresem pozostają m.in. pełny program pentestów,

zaawansowany WAF, dedykowany secret manager oraz własny certyfikat TLS na hoście aplikacji (w produkcji terminacja TLS leży po stronie Cloudflare).

Jakość danych i procesy organizacyjne. Skuteczność katalogu zależy od aktualności wydarzeń wprowadzanych przez organizatorów oraz od moderacji administratora. MVP nie automatyzuje scrapingu zewnętrznych źródeł ani nie gwarantuje kompletności oferty rynkowej — to ograniczenie produktowe wynikające z modelu „organizator jako źródło prawdy", a nie usterka techniczna.

18.3.1 Ograniczenia wydajnościowe (RPS)

W wymaganiach jakościowych MVP zapisano docelowy test obciążenia: **ok. 10 000 rekordów** w katalogu oraz **ok. 50 RPS** przy akceptowalnym P95 odpowiedzi list/detalu. W oddawanej wersji:

- zastosowano mitigacje lokalne (paginacja listy, osobne markery mapy, indeksy / cache tam gdzie uzasadnione);
- **nie wykonano** formalnego load testu 10k/50 RPS (brak raportu RPS/P95 jako artefaktu odbioru);
- środowisko produkcyjne (Oracle Always Free + Cloudflare Tunnel) jest wystarczające do demonstracji i ruchu prezentacyjnego, ale nie zostało zweryfikowane pod kątem założonego progu 50 RPS.

Ograniczenie to jest świadome: priorytetem było domknięcie happy-path funkcjonalnego, bezpieczeństwa podstawowego (JWT/RBAC) i uruchamialności. Pomiar 10k/50 RPS pozostaje w backlogu jakościowym.

18.4 Możliwości dalszego rozwoju

Płatności (PSP), object storage / galeria albumów, watchlista, iCal, waitlista, eksport list startowych, store — por. `docs/pomysly-rozwoju.md`; uzupełnienie testów obciążeniowych RPS.

18.5 Wnioski końcowe

RacePortal stanowi kompletne MVP demonstracyjne katalogu i zapisów motorsportowych z czytelnością architektury, testami funkcjonalnymi i ścieżką prezentacji dla odbiorcy akademickiego oraz biznesowego — przy jasno nazwanych lukach (PSP, hardening uploadów / object storage, load test RPS).

Bibliografia

[1] „RacePortal — repozytorium projektu”. `https://github.com/Gucio-009/RacePortal`. Branch `wojtek`; dostęp: 2026-10-15.

Spis rysunków

4.1 Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny). 32

7.1 Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna). 51

8.1 Diagram klas / model danych (ERD) — wersja koncepcyjna projektu. 53

Spis tabel

4.1 Karta udziałowca: Gość / kibic 33
4.2 Karta udziałowca: Kierowca 33
4.3 Karta udziałowca: Organizator 34
4.4 Karta udziałowca: Administrator 34
4.5 Karta udziałowca: Właściciel toru / partner obiektu 35
4.6 Porównanie modelu grup społecznościowych z modelem RacePortal (ujęcie koncepcyjne). 35

7.1 Zintegrowany ekosystem wydarzeń motorsportowych 45
7.2 Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów 45
7.3 Przejrzysty podział ról w systemie 45
7.4 Publiczny katalog i inteligentne filtrowanie 46
7.5 Szczegóły wydarzenia i wyznaczanie trasy 46
7.6 Zarządzanie profilem i kontem 46
7.7 Wirtualny Garaż Kierowcy 47
7.8 Szybkie aplikowanie na wydarzenia 47
7.9 Skuteczny system powiadomień 47
7.10 Panel Zarządzania dla Organizatora 47
7.11 Panel Administratora i wsparcie użytkowników 48
7.12 Mobilna aplikacja wizytówkowa 48
7.13 Wydajność i responsywność platformy 48
7.14 Dostępność wieloplatformowa (RWD) 49
7.15 Niezawodność i łatwość wdrażania 49
7.16 Bezpieczeństwo danych osobowych i kont 49
7.17 Ścisła kontrola dostępu do zasobów 50
7.18 Odporność na nadużycia platformy 50
7.19 Nowoczesny design branżowy 50
7.20 Intuicyjność i czytelność procesów 51

Załączniki

Płyta CD z następującą zawartością:

- *pliki projektowe* – pliki składające się na całość projektu
  - repozytorium kodu źródłowego wraz z instrukcją zbudowania i uruchomienia projektu
  - źródło pracy inżynierskiej.
- *Otrombke Natalia_Kasprowicz Oliwier_Gutowski Michał_Parkitny Miłosz_Wronisz Wojciech_praca pisemna* – katalog zawierający plik PDF z pracą inżynierską.